Machine Programming

Lecture 19 – PL Kaleidoscope V: Accelerated Computing

Ziyang Li

Logistics

- Oral presentation
  - The papers will be sent out today via email!
  - Attendence will be recorded unless the course staff are notified!
- Final project proposal
  - Final project proposal due this Sunday!
  - Staff will reach out to provide feedback after Sunday, and set up meetings
- Final project presentation
  - Tuesday Dec 22, 6-9pm

Requirements

- **Format:** Write it in **Markdown** and submit the `.md` file.
- **Length: At most one page**, not counting references.
- **Group members:** List the full name and JHU email of every member at the top.
- **Structure:** Include the **three sections** below.

Sections

1. **Motivation.** What problem are you tackling, and why does it matter for machine programming? Explain what your project adds beyond existing work, and briefly mention the closest related papers or tools.
2. **Expected Outcomes.** What do you expect to deliver, and how will you know it works? Describe the metrics, benchmarks, or case studies you will use, and what you will compare against.
3. **Division of Work.** Say briefly which part of the project each member is responsible for.

Template

```
# <Project Title>
**Members:** Alice Smith (asmith1@jh.edu), Bob Lee (blee2@jh.edu), ...

## 1. Motivation
## 2. Expected Outcomes
## 3. Division of Work
```

When submitting on Gradescope, one member uploads the `.md` file and adds all teammates to the submission.

(Old) News bit of the day...



May 14, 2025 Science

AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms

AlphaEvolve team

Share

AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms — Google DeepMind blog, May 2025

May 14, 2025 Science

AlphaEvolve
coding ag
advan

New AI agent evolves algorithms for math and practical applications in computing by combining the creativity of large language models with automated evaluators

Large language models (LLMs) are remarkably versatile. They can summarize documents, generate code or even brainstorm new ideas. And now we've expanded these capabilities to target fundamental and highly complex problems in mathematics and modern computing.

Today, we're announcing AlphaEvolve, an evolutionary coding agent powered by large language models for general-purpose algorithm discovery and optimization. AlphaEvolve pairs the creative problem-solving capabilities of our Gemini models with automated evaluators that verify answers, and uses an evolutionary framework to improve upon the most promising ideas.

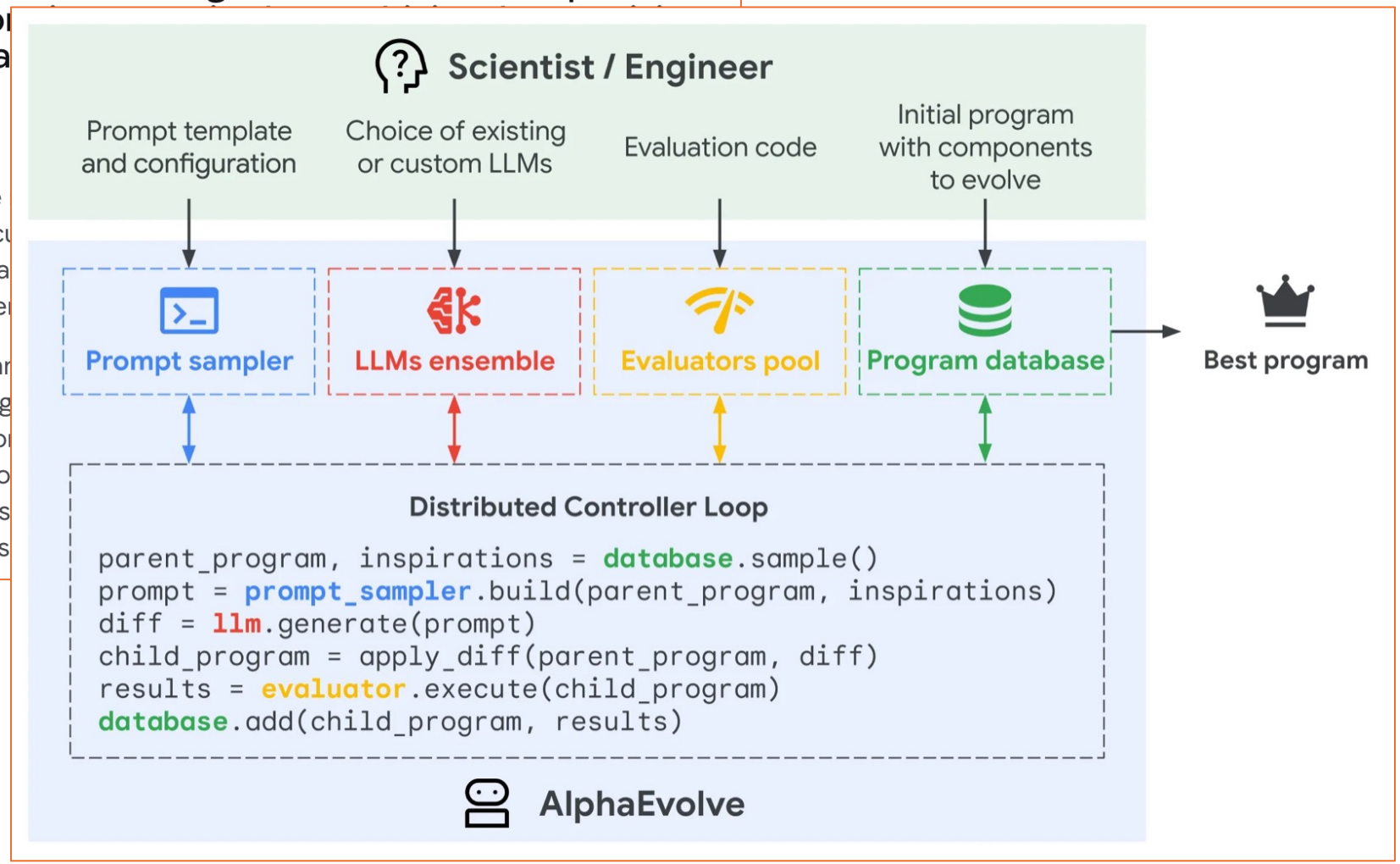
AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms — Google DeepMind blog, May 2025

May 14, 2025 Science

New AI agent evolves algorithms for math and practical applicatio of large la

Large language
summarize docu
now we've expa
complex proble

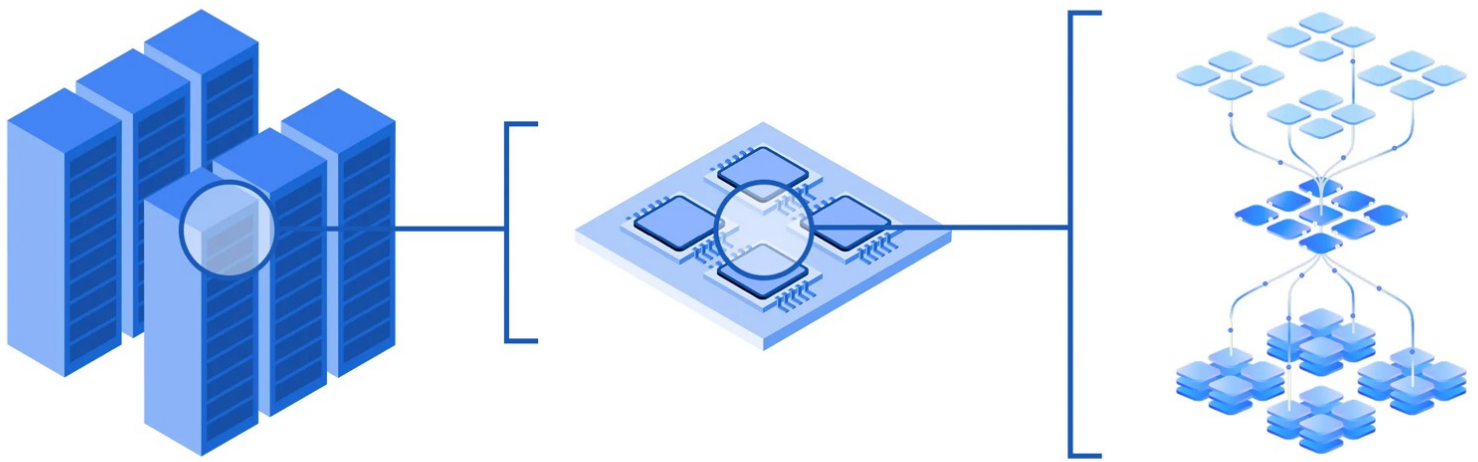
Today, we're ann
powered by larg
and optimizatio
capabilities of o
answers, and us
promising ideas



AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms — Google DeepMind blog, May 2025

Optimizing our computing ecosystem

Over the past year, we've deployed algorithms discovered by AlphaEvolve across Google's computing ecosystem, including our data centers, hardware and software. The impact of each of these improvements is multiplied across our AI and computing infrastructure to build a more powerful and sustainable digital ecosystem for all our users.



Data Center Optimization
Borg Scheduling

Hardware Optimization
TPU Circuit Design

Software Optimization
Gemini Training

AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms — Google DeepMind blog, May 2025

June 7, 2023 Science

AlphaDev discovers faster sorting algorithms

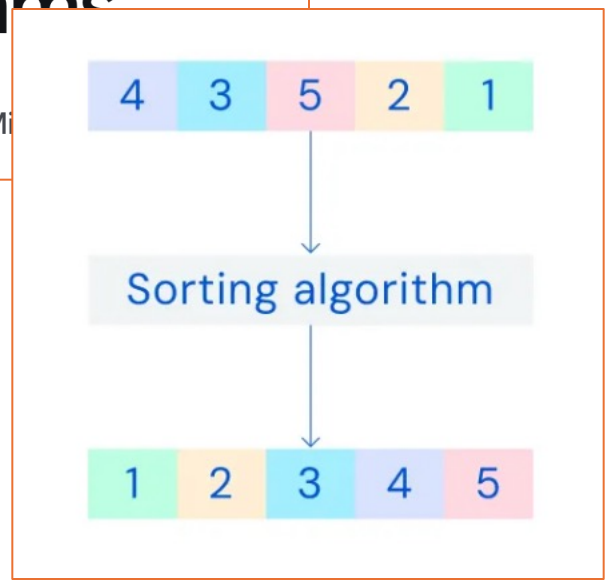
Daniel J. Mankowitz and Andrea Michi

Mankowitz et al., Faster sorting algorithms discovered using deep reinforcement learning (AlphaDev), Nature 2023

June 7, 2023 Science

AlphaDev discovers faster sorting algorithms

Daniel J. Mankowitz and Andrea Mi

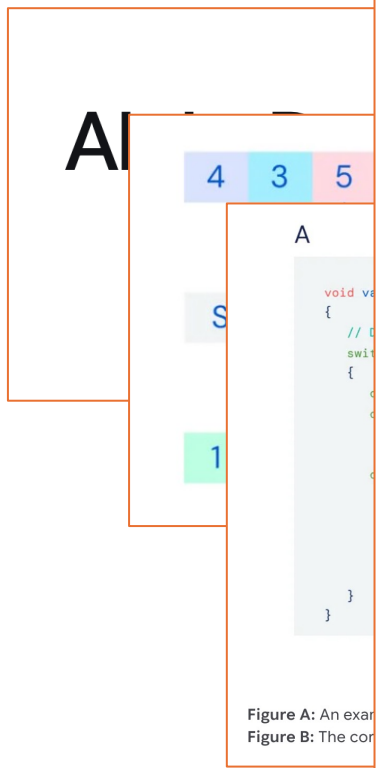


Mankowitz et al., Faster sorting algorithms discovered using deep reinforcement learning (AlphaDev), Nature 2023



Figure A: An example C++ algorithm that sorts up to two elements.
Figure B: The corresponding assembly representation of the code.

Mankowitz et al., Faster sorting algorithms discovered using deep reinforcement learning (AlphaDev), Nature 2023



Original

```
Memory[0] = A
Memory[1] = B
Memory[2] = C
Memory[3] = D

mov Memory[0] P  // P = A
mov Memory[1] Q  // Q = B
mov Memory[2] R  // R = C
mov Memory[3] S  // S = D

cmp S P
mov P T
cmovl S P  // P = min(A, D)
cmovl T S  // S = max(A, D)
cmp R P
mov P T
cmovg R P  // P = max(C, min(A, D))
cmovl R T  // T = min(A, C, D)
cmp Q T
mov T U
cmovl Q U  // U = min(A, B, C, D)
cmovl T Q  // Q = max(B, min(A, C, D))

mov U Memory[0]  // = min(A, B, C, D)
mov Q Memory[1]  // = max(B, min(A, C, D))
mov P Memory[2]  // = max(C, min(A, D))
mov S Memory[3]  // = max(A, D)
```

AlphaDev

```
Memory[0] = A
Memory[1] = B
Memory[2] = C
Memory[3] = D

mov Memory[0] P  // P = A
mov Memory[1] Q  // Q = B
mov Memory[2] R  // R = C
mov Memory[3] S  // S = D

cmp S P
mov P T
cmovl S P  // P = max(C, min(A, D))
cmovl T S  // S = max(A, D)
cmp R P

cmovg R P  // P = max(C, min(A, D))
cmovl R T  // T = min(A, C)
cmp Q T
mov T U
cmovl Q U  // U = min(A, B, C)
cmovl T Q  // Q = max(B, min(A, C))

mov U Memory[0]  // = min(A, B, C, D)
mov Q Memory[1]  // = max(B, min(A, C))
mov P Memory[2]  // = max(C, min(A, D))
mov S Memory[3]  // = max(A, D)
```

Left: The original implementation with max (B, min (A, C, D))used in a larger sorting algorithm for sorting eight elements.
Right: AlphaDev discovered that only max (B, min (A, C)) is needed when using its copy move.

Mankowitz et al., Faster sorting algorithms discovered using deep reinforcement learning (AlphaDev), Nature 2023

Google DeepMind

AlphaEvolve: A coding agent for scientific and algorithmic discovery

Alexander Novikov*, Ngân Vũ*, Marvin Eisenberger*, Emilien Dupont*, Po-Sen Huang*, Adam Zsolt Wagner*, Sergey Shirobokov*, Borislav Kozlovskii*, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli and Matej Balog*
Google DeepMind[1]

In this white paper, we present *AlphaEvolve*, an evolutionary coding agent that substantially enhances capabilities of state-of-the-art LLMs on highly challenging tasks such as tackling open scientific problems or optimizing critical pieces of computational infrastructure. *AlphaEvolve* orchestrates an autonomous pipeline of LLMs, whose task is to improve an algorithm by making direct changes to the code. Using an evolutionary approach, continuously receiving feedback from one or more evaluators, *AlphaEvolve* iteratively improves the algorithm, potentially leading to new scientific and practical discoveries. We demonstrate the broad applicability of this approach by applying it to a number of important computational problems. When applied to optimizing critical components of large-scale computational stacks at Google, *AlphaEvolve* developed a more efficient scheduling algorithm for data centers, found a functionally equivalent simplification in the circuit design of hardware accelerators, and accelerated the training of the LLM underpinning *AlphaEvolve* itself. Furthermore, *AlphaEvolve* discovered novel, provably correct algorithms that surpass state-of-the-art solutions on a spectrum of problems in mathematics and computer science, significantly expanding the scope of prior automated discovery methods (Romera-Paredes et al., 2023). Notably, *AlphaEvolve* developed a search algorithm that found a procedure to multiply two 4×4 complex-valued matrices using 48 scalar multiplications; offering the first improvement, after 56 years, over Strassen's algorithm in this setting. We believe *AlphaEvolve* and coding agents like it can have a significant impact in improving solutions of problems across many areas of science and computation.

Novikov et al., AlphaEvolve: A coding agent for scientific and algorithmic discovery, Google DeepMind 2025 — arXiv:2506.13131

Google DeepMind

2026-08-18

Improving the matrix multiplication exponent with modern optimization and AlphaEvolve

Emilien Dupont[*,1], **Marvin Eisenberger**[*,1], **Borislav Kozlovskii**[*,1], **Abbas Mehrabian**[*,1], **Francisco J. R. Ruiz**[*,1], **Abigail See**[*,1], **Renfei Zhou**[*,2], **Josh Alman**[3], **Virginia Vassilevska Williams**[4] **and Matej Balog**[1]

[*]Equal contribution in alphabetical order, [1]Google DeepMind, [2]Carnegie Mellon University, [3]Columbia University, [4]MIT

The current best bounds on the matrix multiplication exponent ω are obtained through a refinement of the laser method called combination loss analysis (Duan et al., 2023; Williams et al., 2024; Alman et al., 2025). In this note, we address the optimization problem at the core of this approach and propose several improvements. First, we reformulate the optimization problem allowing us to solve it in a larger setting than was previously possible. Second, we leverage recent advances in machine learning to design a new optimization algorithm for this problem. Finally, we refine the resulting optimization algorithm with AlphaEvolve. Our combined approach yields an upper bound of $\omega < 2.371177$, improving the previous best bound of 2.371339**.**

Dupont et al., Improving the matrix multiplication exponent with modern optimization and AlphaEvolve, 2026 — arXiv:2608.16884

Google DeepMind

2026-08-18

Improving the matrix multiplication exponent with modern optimization and AlphaEvolve

Emilien Dupont[*,1], Mar
Abigail See[*,1], Renfei Zl
[*]Equal contribution in alpha

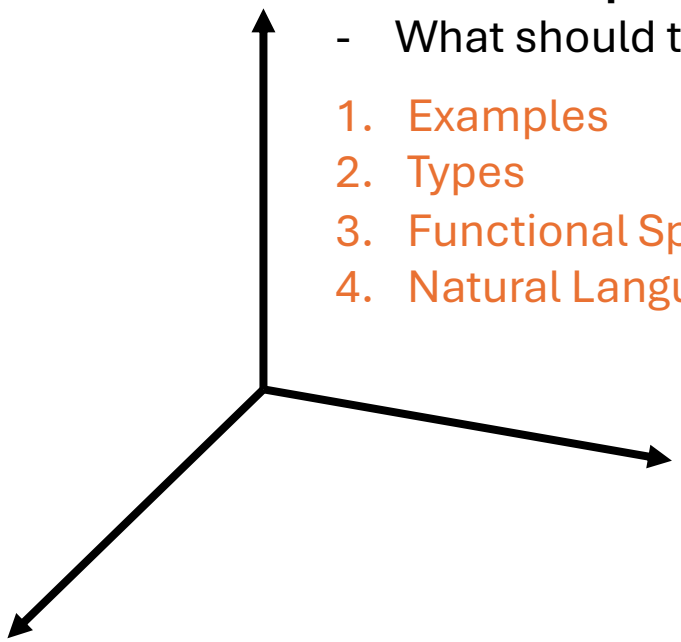
The current best bou
of the laser method
et al., 2025). In this
several improvement
setting than was prev
a new optimization a
with AlphaEvolve. O
previous best bound

Here, we improve the bound to $\omega < 2.371\mathbf{177}$ using a two-step approach. First, we leverage recent advances in machine learning and adjacent areas to address the non-convex optimization problem of combination loss analysis using a gradient descent approach; this alone improves the previous state-of-the-art (SOTA) bound by $\approx 0.97 \times 10^{-4}$. Second, we use AlphaEvolve (Novikov et al., 2025) to improve our optimization algorithm; this raises the improvement over the SOTA to $\approx 1.62 \times 10^{-4}$.

Duan et al. (2023)	2.371**866**
Williams et al. (2024)	2.371**552**
Alman et al. (2025)	2.371**339**
This note	2.371**177**

Table 1 | Recent improvements to ω.

The Course So Far



Behavioral Specification
- What should the program do?

1. Examples
2. Types
3. Functional Specifications
4. Natural Language

Structural Specification
- What is the space of the programs?

General Purpose Programming Language
Python / Java / C / Rust / …

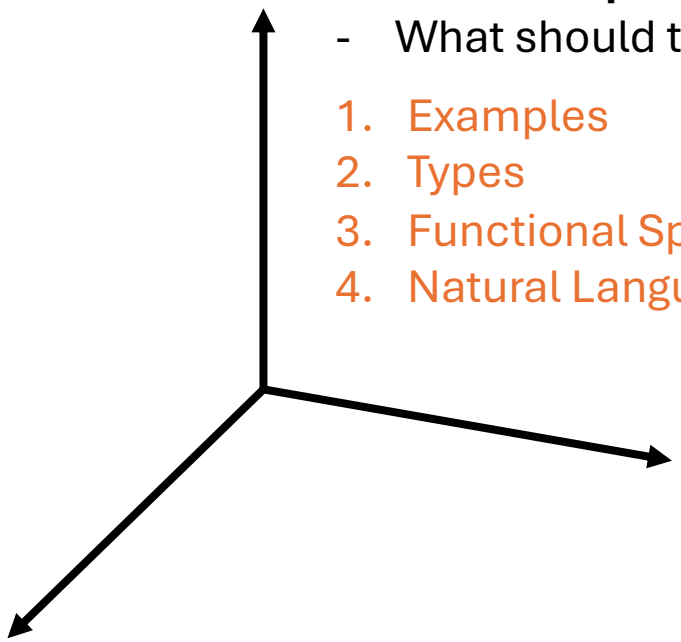
Custom Domain Specific Languages

Synthesis Strategy
- How do we find such a program?

Enumeration: Bottom-up and Top-Down

Programming with Large Language Models
- Next token prediction, prompting, controlled decoding
- Iterative refinement, agentic frameworks and tool use
- Pre-training, fine-tuning, reinforcement learning

Considering software quality...



Behavioral Specification

- What should the program do?

1. Examples
2. Types
3. Functional Specifications
4. Natural Language

Correctness Specifications

Structural Specification

- What is the space of the programs?

General Purpose Programming Language

Python / Java / C / Rust / ...

Fixed

Custom Domain Specific Languages

Synthesis Strategy

- How do we find such a program?

Enumeration: Bottom-up and Top-Down

Programming with Large Language Models

- Next token prediction, prompting, controlled decoding
- Iterative refinement, agentic frameworks and tool use
- Pre-training, fine-tuning, reinforcement learning

Fixed

Considering software quality...

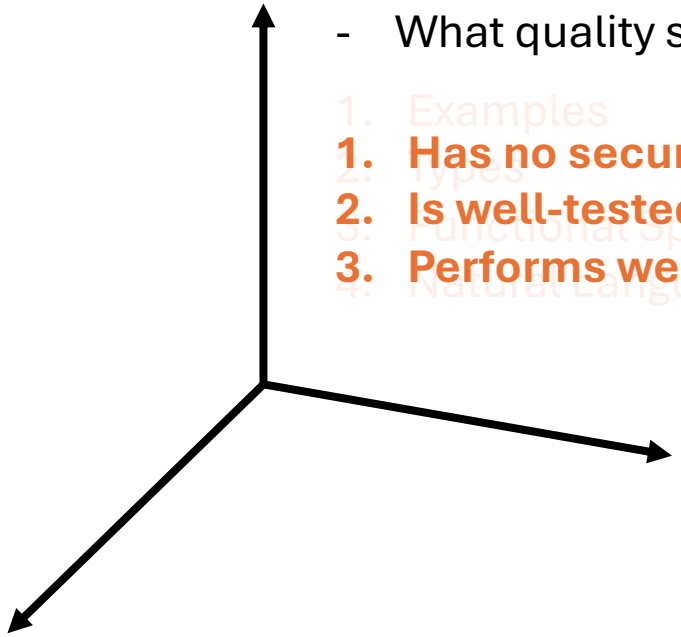
Quality Specifications

Behavioral Specification

- What quality should programs have?

1. Examples
2. Types
3. Functional Specifications
4. Natural Language

1. **Has no security flaw** (safety, privacy, etc.)
2. **Is well-tested** (no regression, good coverage, etc.)
3. **Performs well** (fast, use less resource, etc.)



Synthesis Strategy

- How do we find such a program?

Enumeration: Bottom-up and Top-Down

Programming with Large Language Models

- Next token prediction, prompting, controlled decoding
- Iterative refinement, agentic frameworks and tool use
- Pre-training, fine-tuning, reinforcement learning

Fixed

Structural Specification

- What is the space of the programs?

General Purpose Programming Language

Python / Java / C / Rust / ...

Custom Domain Specific Languages

Fixed

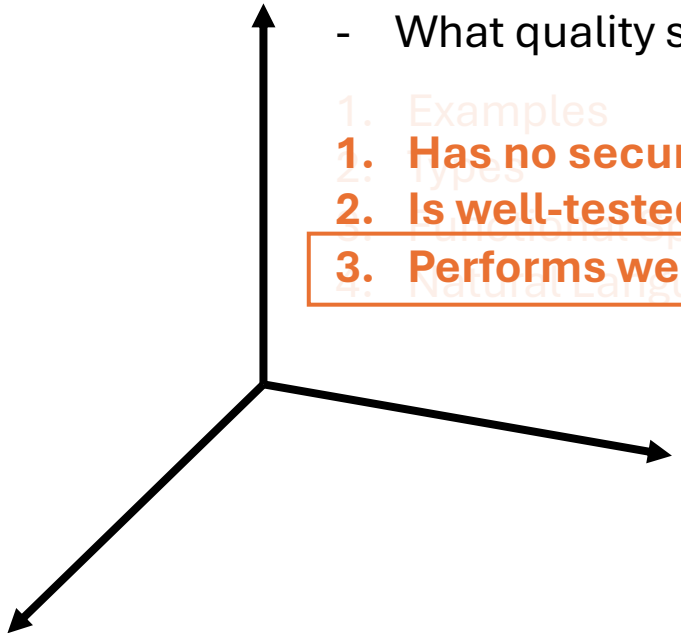
Considering software quality...

Quality Specifications

Behavioral Specification
- What quality should programs have?

1. Examples
2. Types
3. Functional Specifications
4. Natural Language

1. **Has no security flaw** (safety, privacy, etc.)
2. **Is well-tested** (no regression, good coverage, etc.)
3. **Performs well** (fast, use less resource, etc.)



Synthesis Strategy
- How do we find such a program?

Enumeration: Bottom-up and Top-Down

Programming with Large Language Models
- Next token prediction, prompting, controlled decoding
- Iterative refinement, agentic frameworks and tool use
- Pre-training, fine-tuning, reinforcement learning

Fixed

Structural Specification
- What is the space of the programs?

General Purpose Programming Language
Python / Java / C / Rust / ...

Custom Domain Specific Languages

Fixed

Accelerated Computing



Accelerated Computing



P P^*

Success Criteria:

- $P \cong P^*$: the two programs are equivalent*
- P^* is safe
- P^* performs better than P

“Equivalent”



Success Criteria:
- $P \cong P^*$: the two programs are equivalent*
- P^* is safe
- P^* performs better than P

- “Completely the same”
  - Observational equivalence: same effect on all possible input
- “Allow some leeway”
  - Conditional equivalence: same effect on inputs satisfying some spec
  - Equivalence on finite input set: same effect on a finite set of inputs

“Safe”



Success Criteria:
- $P \cong P^*$: the two programs are equivalent*
- P^* is safe
- P^* performs better than P

- Memory Safety
  - No concurrent read/write; race condition & data corruption
- Thread Safety
  - No non-determinism; no deadlock

“Performs Better”



Success Criteria:
- $P \cong P^*$: the two programs are equivalent*
- P^* is safe
- P^* performs better than P

- Wall-clock
  - Measured against wall-clock; might differ across different machine and configurations
- CPU/GPU Cycle
  - Measured against amount of computation on chip
- Memory usage
  - Measured against peak memory usage in space

Problem Setup in Accelerated Computing



Problem Setup in Accelerated Computing

1. CPU Program optimization / Transpilation



CPU Implementation
(C/C++/Python/Java/Rust/MATLAB)

CPU Implementation
(C/C++/Python/Java/Rust/MATLAB)

Problem Setup in Accelerated Computing

2. Hardware-accelerated computing



CPU Implementation
(C/C++/Python/Java/Rust/MATLAB)

GPU/TPU Implementation
(C++ + CUDA/HIP/Kokkos/Alpaca)

Problem Setup in Accelerated Computing

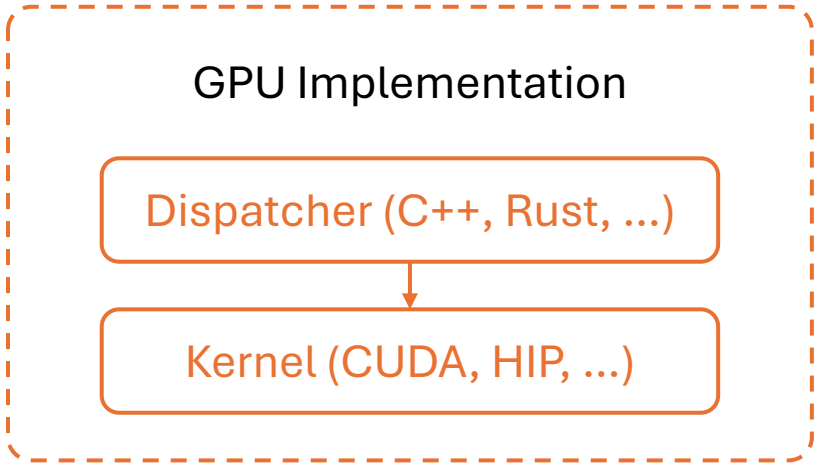
3. Accelerated Computing Program Optimization



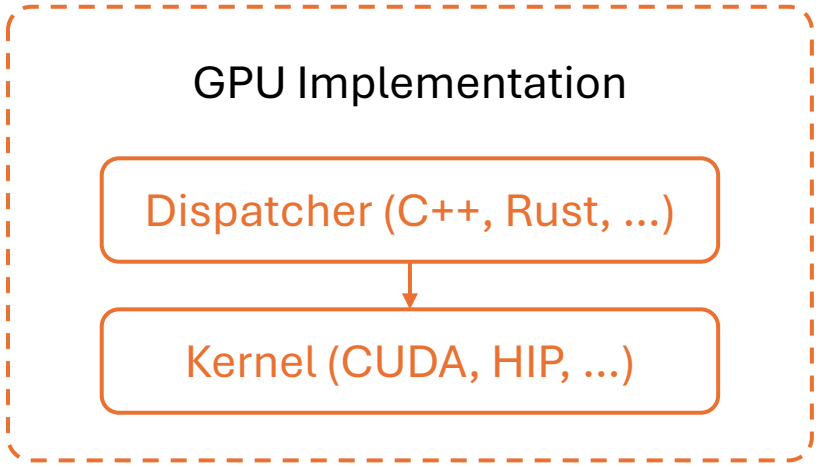
GPU/TPU Implementation
(C++ + CUDA/HIP/Kokkos/Alpaca)

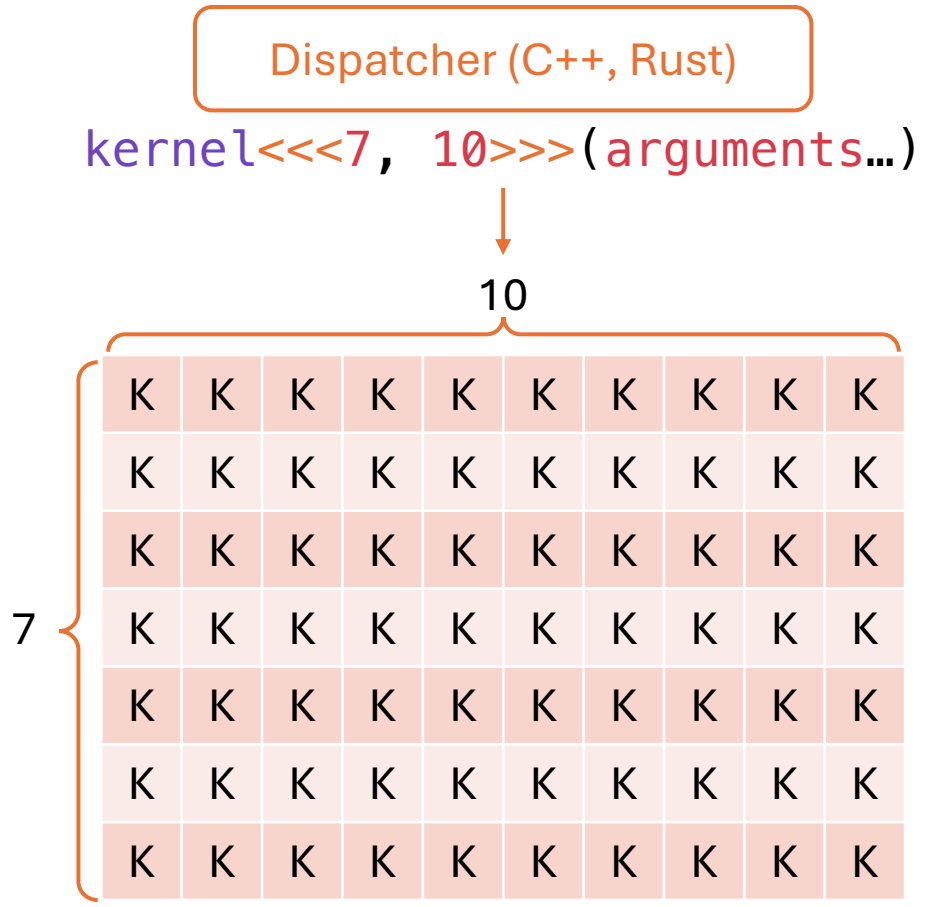
GPU/TPU Implementation
(C++ + CUDA/HIP/Kokkos/Alpaca)

What it means for a “GPU implementation”

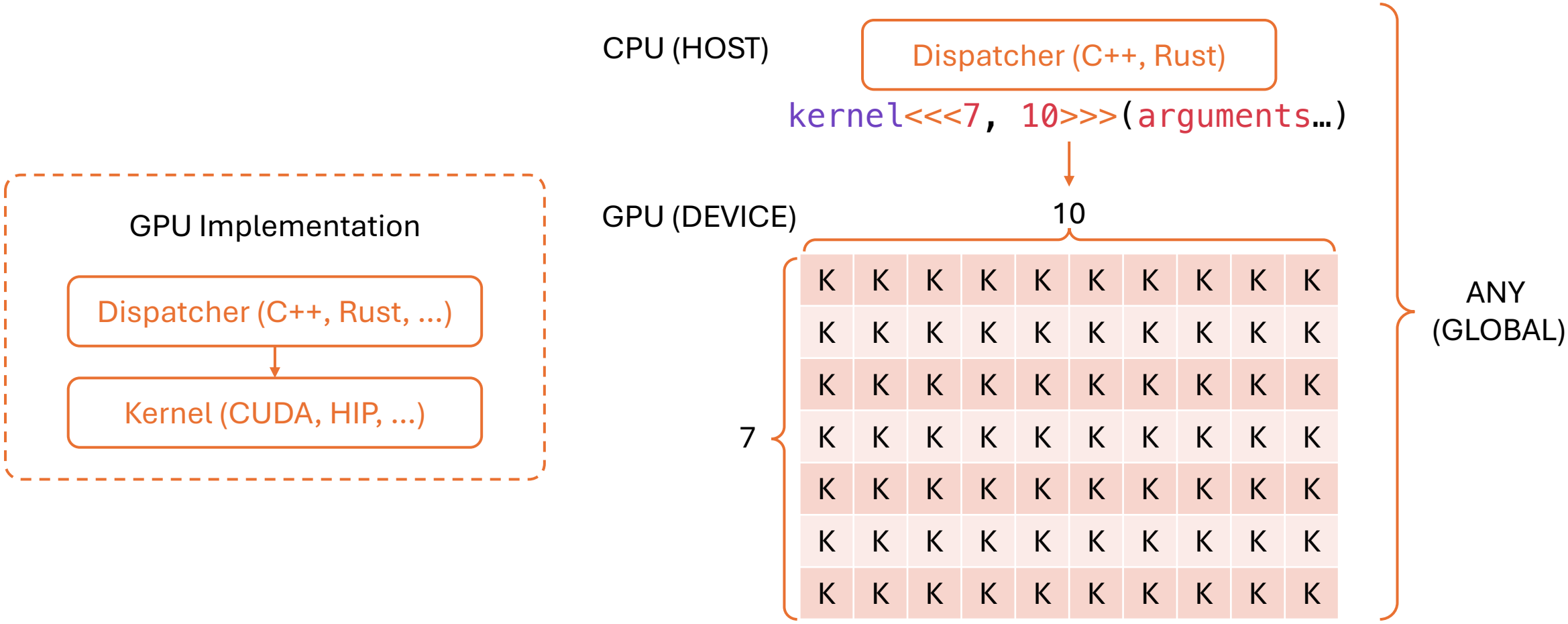


What it means for a “GPU implementation”

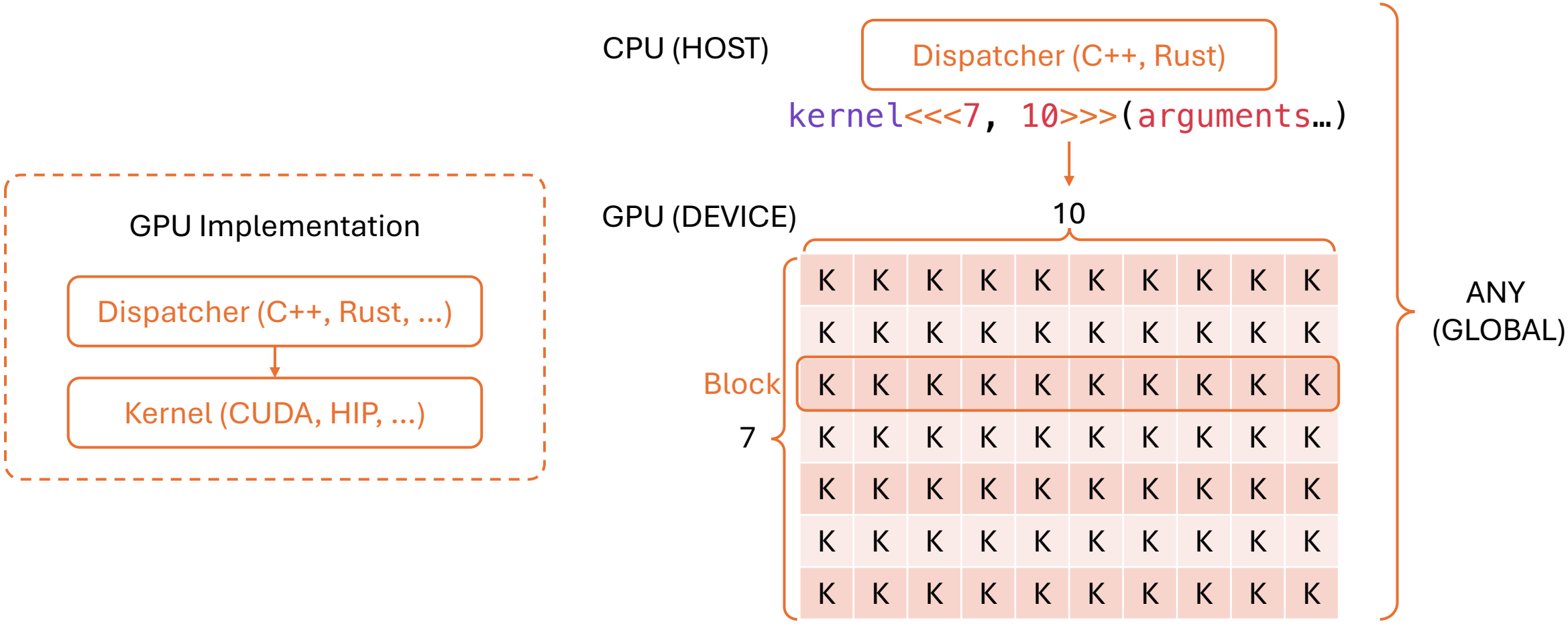




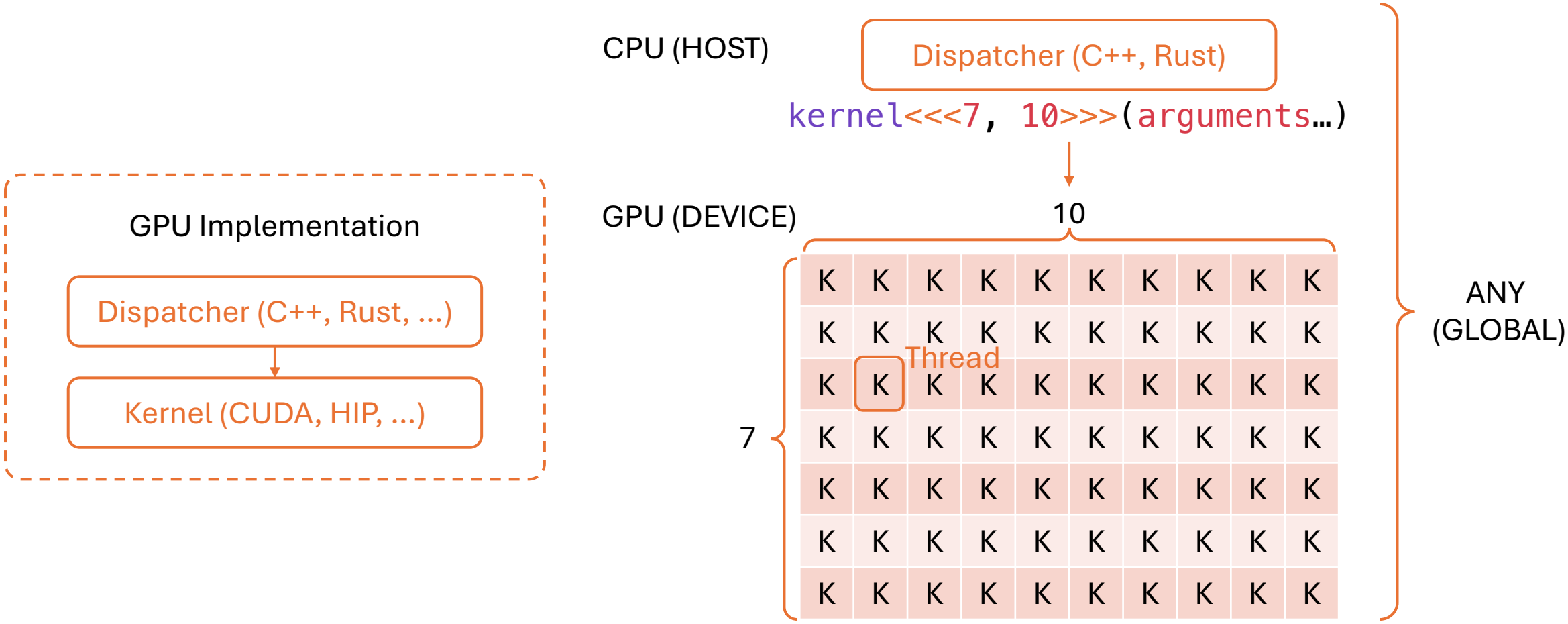
What it means for a “GPU implementation”



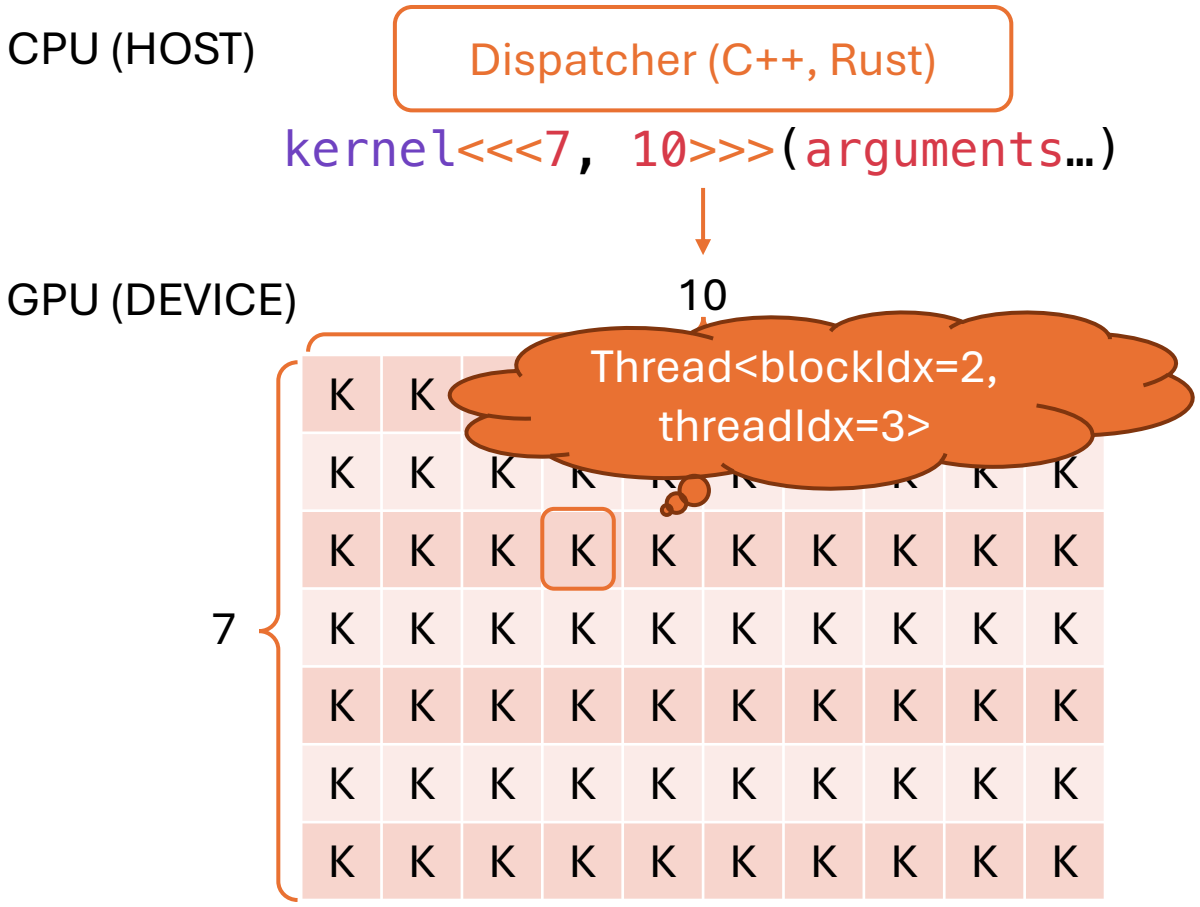
What it means for a “GPU implementation”



What it means for a “GPU implementation”



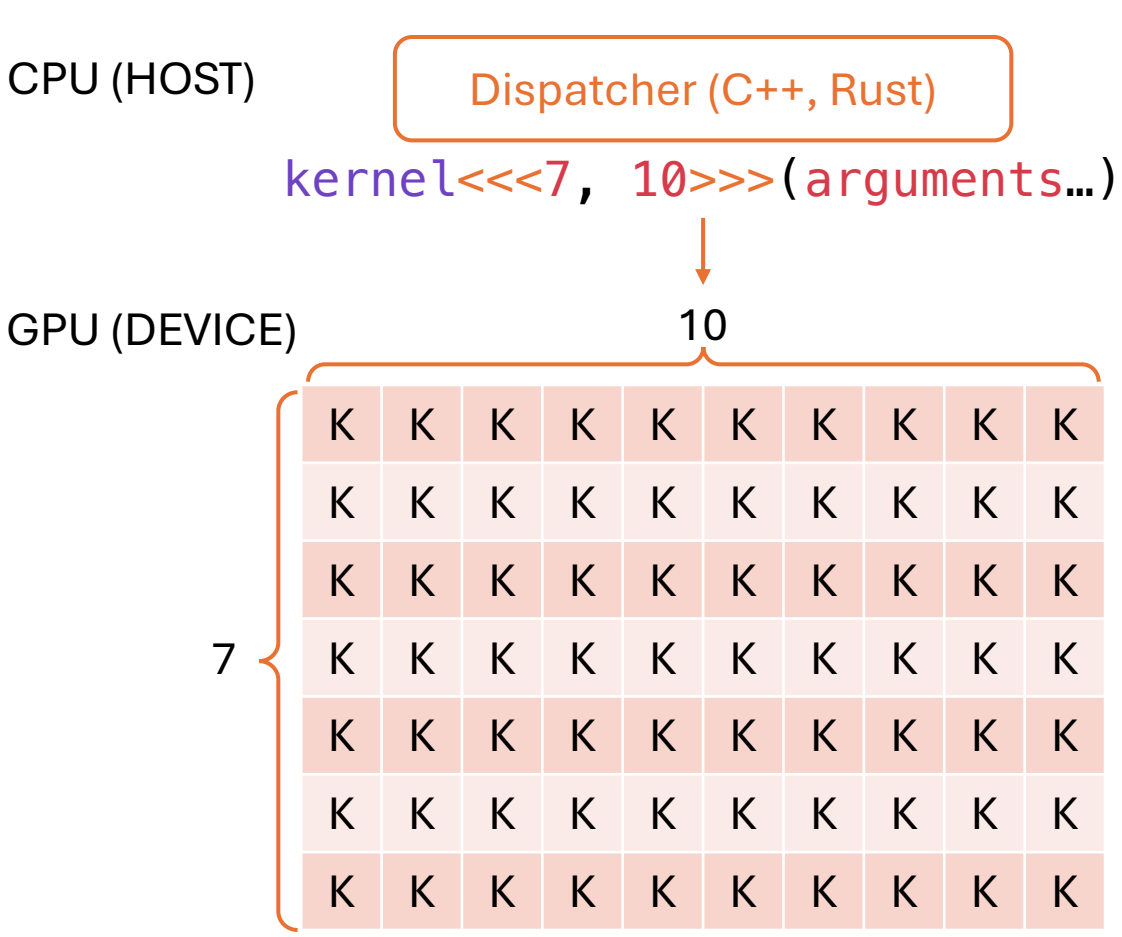
What it means for a “GPU implementation”



Key idea:

- All the spawned kernel threads accept exactly the same input arguments passed from the CPU
- The differentiating factor would be “blockIdx” and “threadIdx”
- Each kernel thread will access the same input arguments but dependent on the different block ID and thread ID
- Each kernel dispatch involves

What it means for a “GPU implementation”



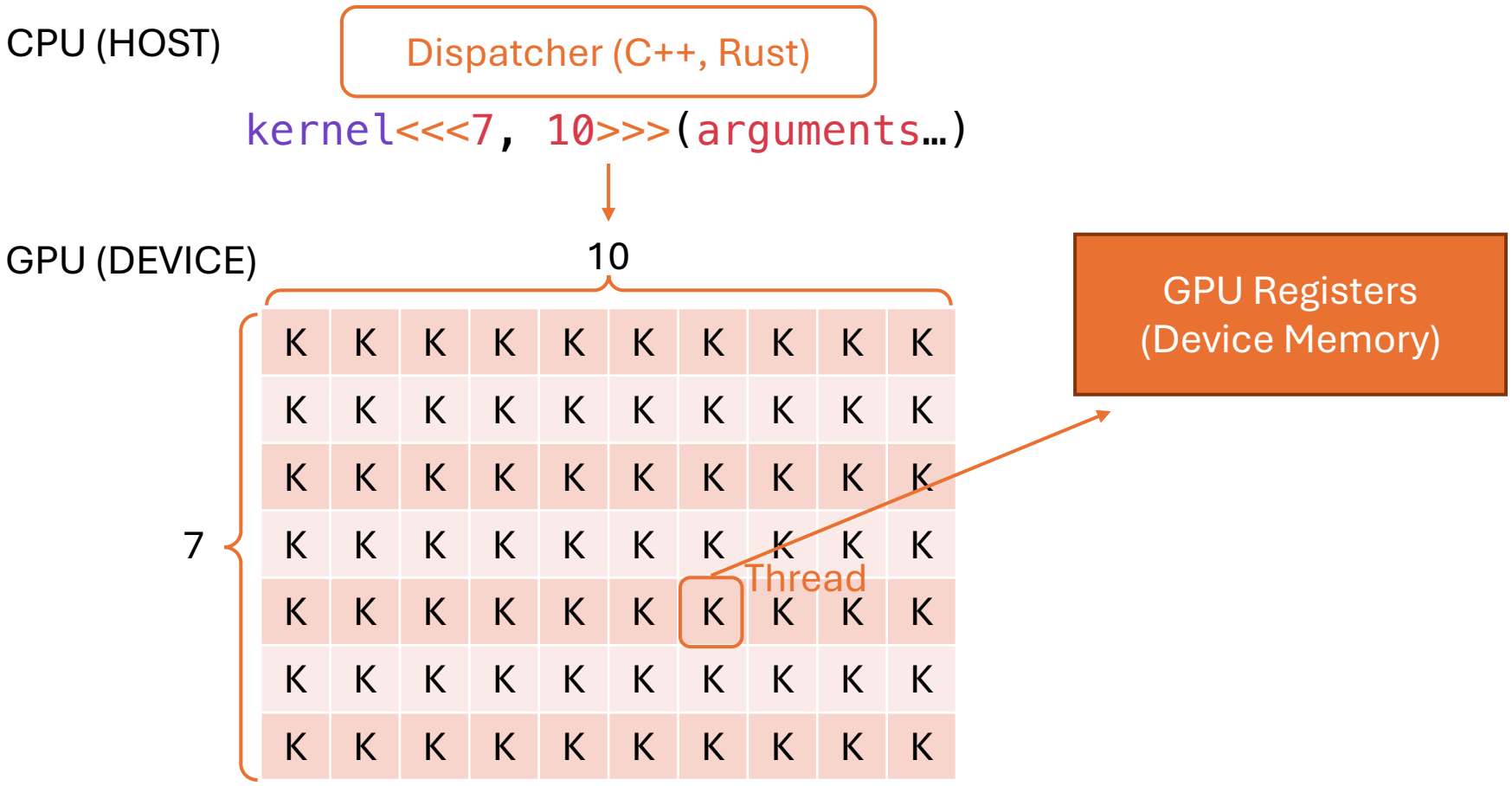
CPU Memory
(Host Memory)

GPU Global Memory
(Device Memory)

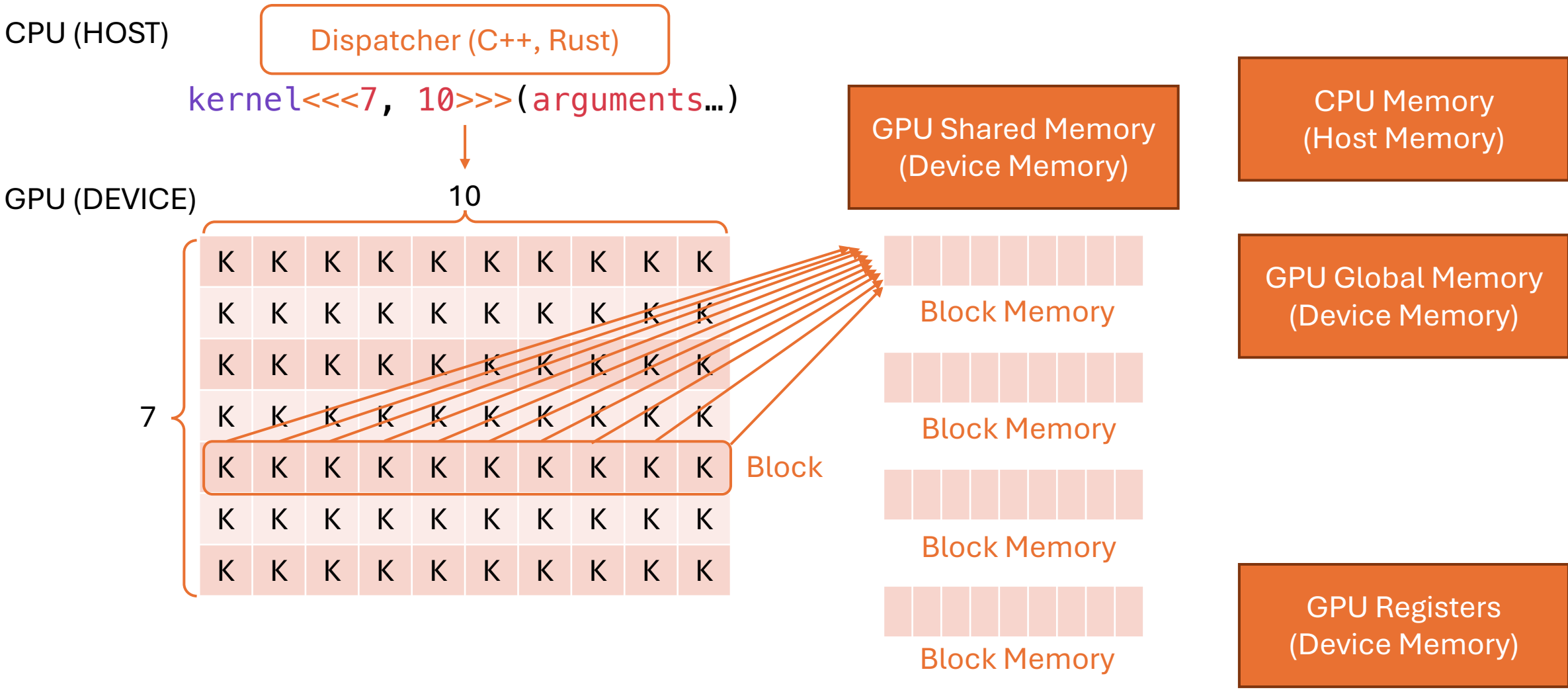
GPU Shared Memory
(Device Memory)

GPU Registers
(Device Memory)

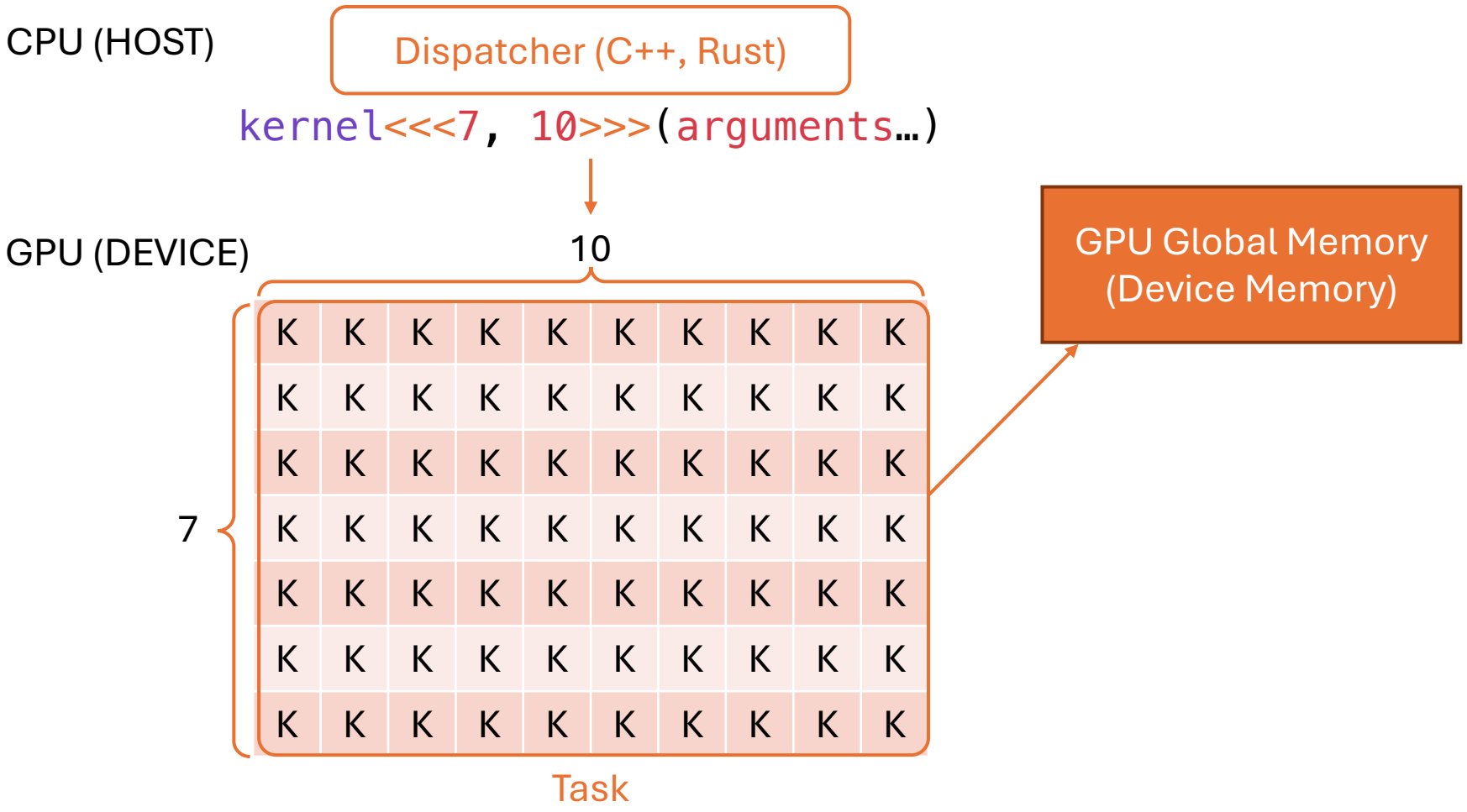
What it means for a “GPU implementation”



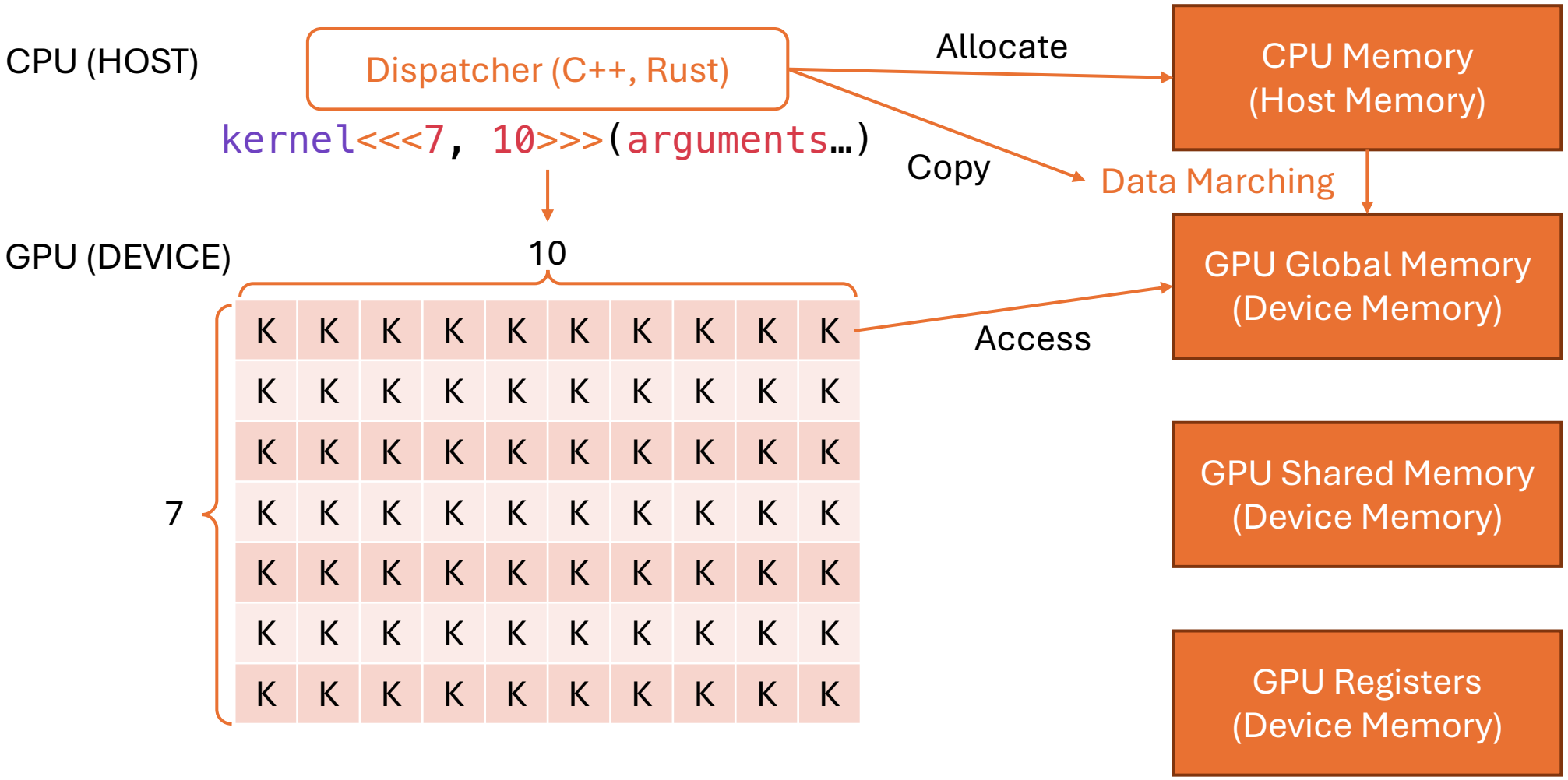
What it means for a “GPU implementation”



What it means for a “GPU implementation”



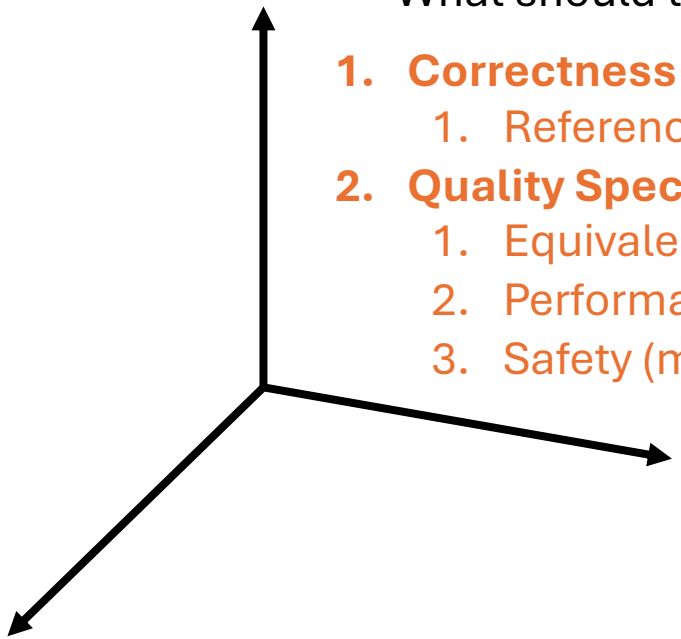
What it means for a “GPU implementation”



Accelerated Computing

Behavioral Specification
- What should the program do?

1. **Correctness Specification**
    1. Reference solution
2. **Quality Specification**
    1. Equivalent
    2. Performance improvement
    3. Safety (memory, thread, etc.)



Synthesis Strategy
- How do we find such a program?

Enumeration
- Bottom-up and Top-Down
- Mutation + Genetic Algorithms

Programming with Large Language Models
- Agentic search

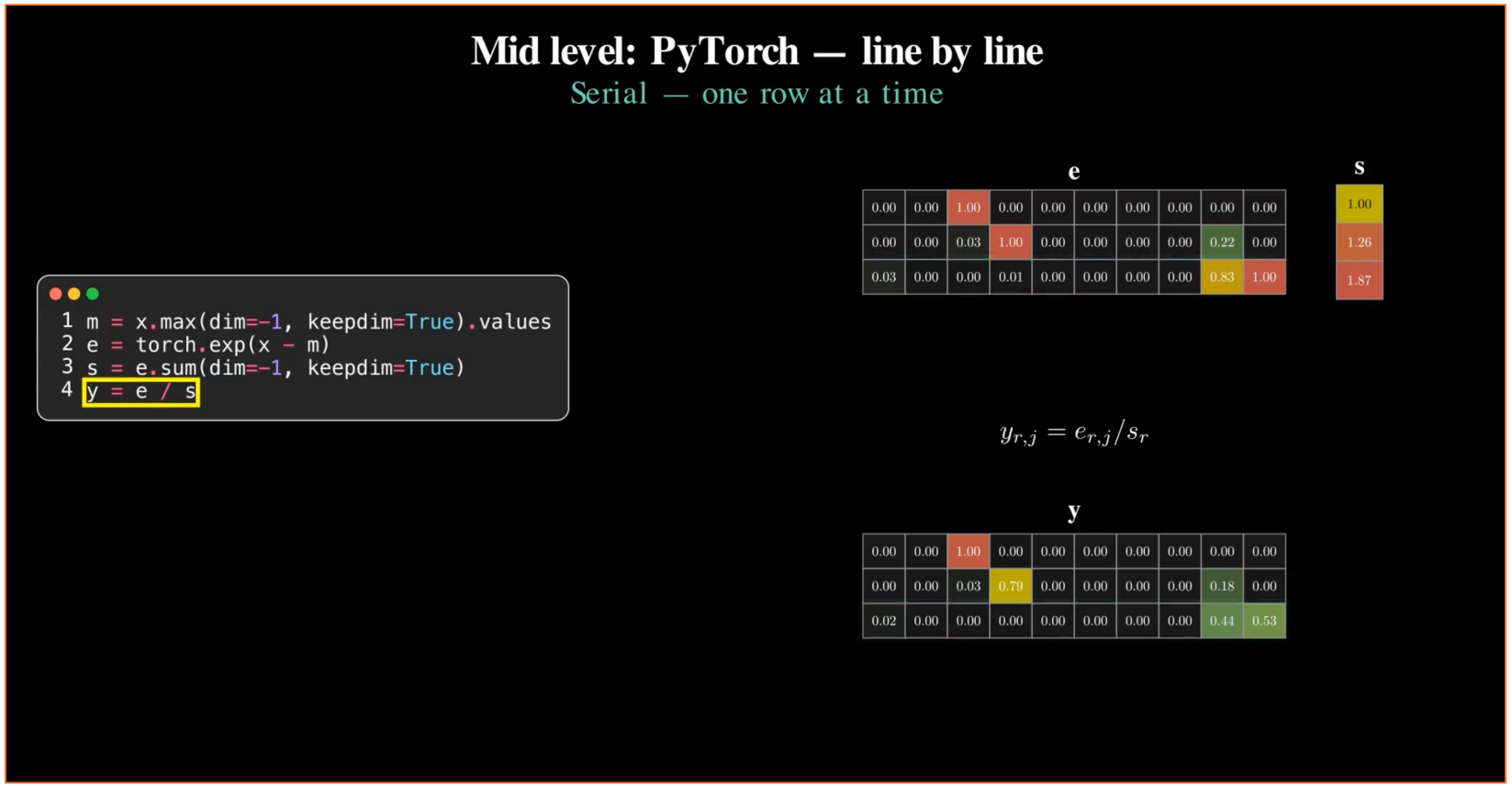
Structural Specification
- What is the space of the programs?

Accelerated Programming Languages
- Low-level/System Programming Languages: Rust/C
- Hardware-accelerated Programming Languages: CUDA/HIP/MLIR
- Embedded Frameworks: Kokkos/Alpaca

Agenda

- CUDA Crash Course: a systematic study on “Softmax”
- Discussion of design spaces for accelerated programming
- Recent works on coding agents + accelerated programming

Demo


Mid level: PyTorch — line by line
Serial — one row at a time
1 m = x.max(dim=-1, keepdim=True).values
2 e = torch.exp(x - m)
3 s = e.sum(dim=-1, keepdim=True)
4 y = e / s
e
0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.03 1.00 0.00 0.00 0.00 0.00 0.22 0.00
0.03 0.00 0.00 0.01 0.00 0.00 0.00 0.00 0.83 1.00
s
1.00
1.26
1.87
$y_{r,j} = e_{r,j}/s_r$
y
0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.03 0.79 0.00 0.00 0.00 0.00 0.18 0.00
0.02 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.44 0.53

Problem Setup in Accelerated Computing

4. Kernel Fusion



GPU/TPU Implementation
With Multiple Individual Kernels

GPU/TPU Implementation
With a Single Fused Kernel

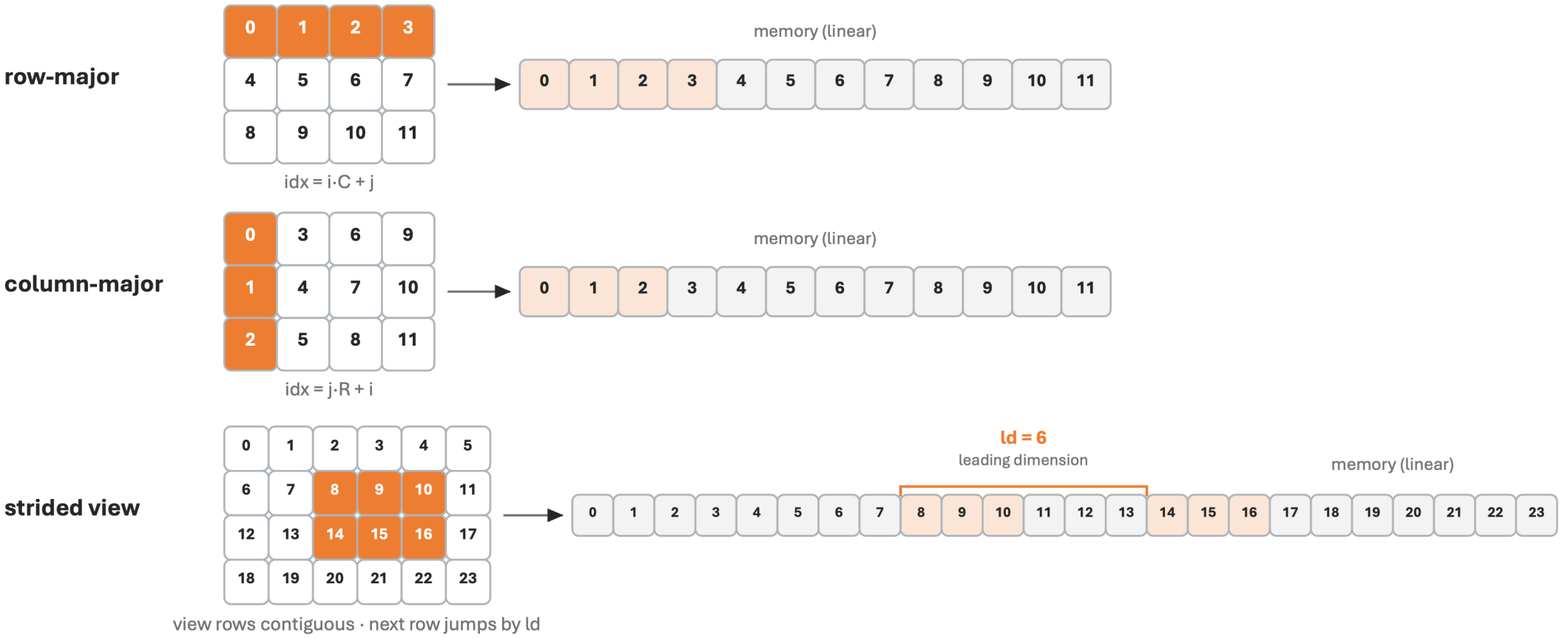
Agenda

- ~~CUDA Crash Course: a systematic study on “Softmax”~~
- Discussion of design spaces for accelerated programming
- Applications of coding agents + accelerated programming

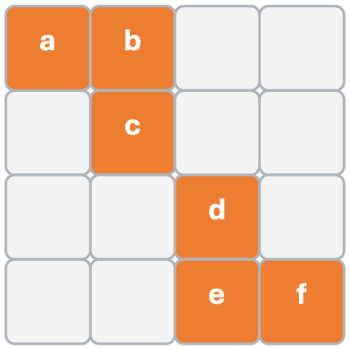
Design Space

- Kernel memory access pattern
- Data structure
- Dispatch engineering

Dense matrix layouts (3 × 4)



Sparse matrix formats



4×4, 6 nonzeros

COO

coordinate list

row	0	0	1	2	3	3

col	0	1	1	2	2	3

val	a	b	c	d	e	f

CSR

compressed sparse row

row_ptr	0	2	3	4	6

col	0	1	1	2	2	3

val	a	b	c	d	e	f

CSC

compressed sparse column

col_ptr	0	1	3	5	6

row	0	0	1	2	3	3

val	a	b	c	d	e	f

ELL

ELLPACK

col_idx	val
0 1	a b
1 ·	c ·
2 ·	d ·
2 3	e f

pad rows → max-nnz

BSR

block sparse row

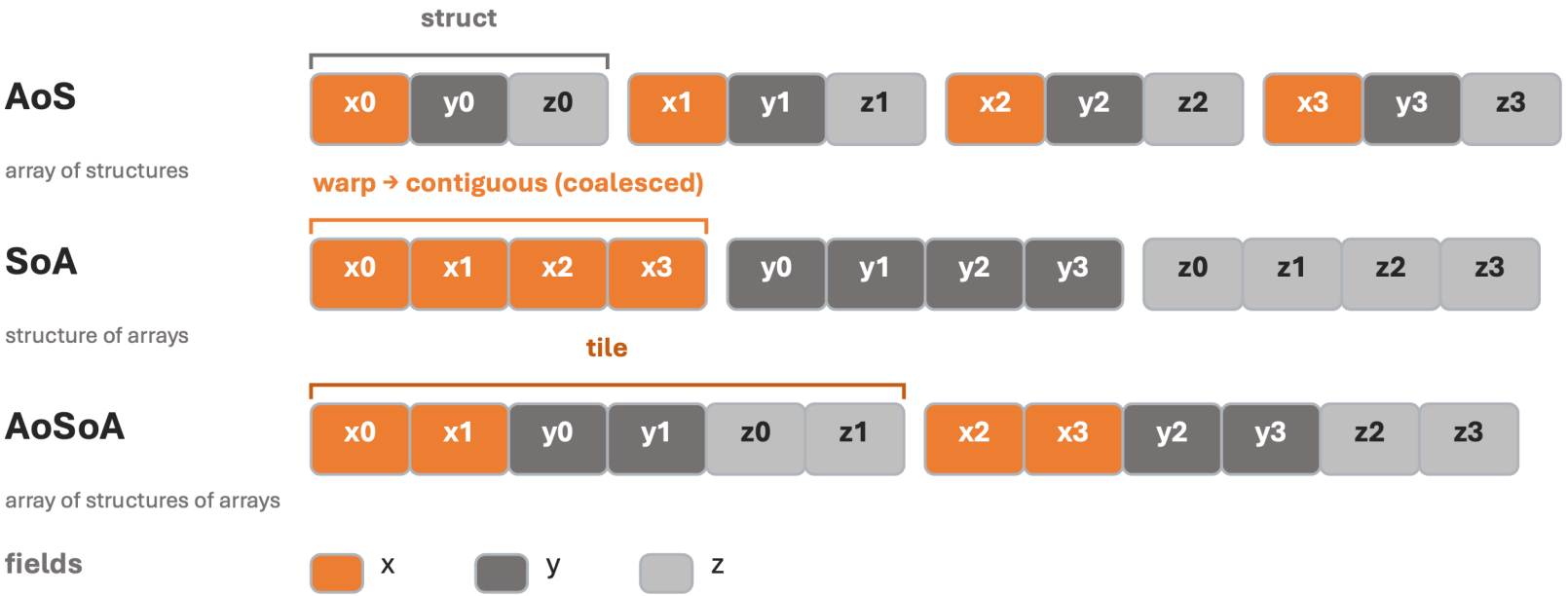
2×2 blocks

a	b
·	c

d	·
e	f

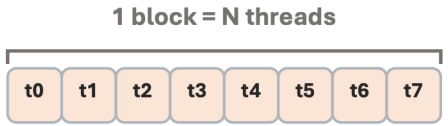
store dense blocks

Memory layout for N particles, 3 fields each



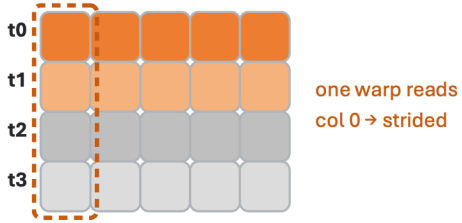
Dispatch engineering: how work maps onto the grid

1 thread / element



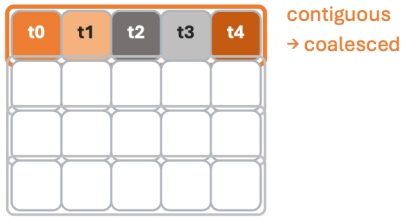
elementwise · one element each

1 thread / row



naive · uncoalesced

1 block / row



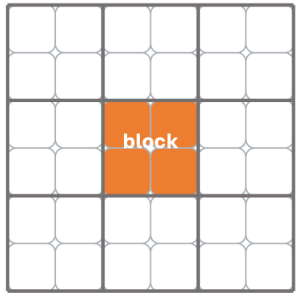
cooperative reduce · coalesced

grid-stride loop



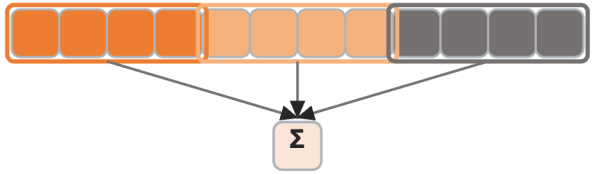
data-size independent · occupancy

2-D tiles



matmul · stencils · 2D grid

split-K



few rows, big dim · split the reduction

Agenda

- ~~CUDA Crash Course: a systematic study on “Softmax”~~
- ~~Discussion of design spaces for accelerated programming~~
- Applications of coding agents + accelerated programming

KernelBench: Can LLMs Write Efficient GPU Kernels?

Anne Ouyang[1,*], Simon Guo[1,*], Simran Arora[1], Alex L. Zhang[2], William Hu[1], Christopher Ré[1], and Azalia Mirhoseini[1]

[1]Stanford University
[2]Princeton University

February 18, 2025

Abstract

Efficient GPU kernels are crucial for building performant machine learning architectures, but writing them is a time-consuming challenge that requires significant expertise; therefore, we explore using language models (LMs) to automate kernel generation. We introduce **KernelBench**, an open-source framework for evaluating LMs' ability to write fast and correct kernels on a suite of 250 carefully selected PyTorch ML workloads. KernelBench represents a real-world engineering environment and making progress on the introduced benchmark directly translates to faster practical kernels. We introduce a new evaluation metric fast_p, which measures the percentage of generated kernels that are functionally correct and offer a speedup greater than an adjustable threshold p over baseline. Our experiments across various state-of-the-art models and test-time methods show that frontier reasoning models perform the best out of the box but still fall short overall, matching the PyTorch baseline in less than 20% of the cases. While we show that results can improve by leveraging execution and profiling feedback during iterative refinement, KernelBench remains a challenging benchmark, with its difficulty increasing as we raise speedup threshold p.

KernelBench: Can LLMs Write Efficient GPU Kernels?

Anne Ouyang[1,*], Simon Guo[1,*], Simran Arora[1], Alex L. Zhang[2], William Hu[1], Christopher Ré[1], and Azalia Mirhoseini[1]

[1]Stanford University
[2]Princeton University

Efficient GPU kernels are c
them is a time-consuming challe
models (LMs) to automate ker
for evaluating LMs' ability to
ML workloads. KernelBench re
introduced benchmark directly
$fast_p$, which measures the perce
greater than an adjustable th
models and test-time methods s
fall short overall, matching the
can improve by leveraging exe
remains a challenging benchma

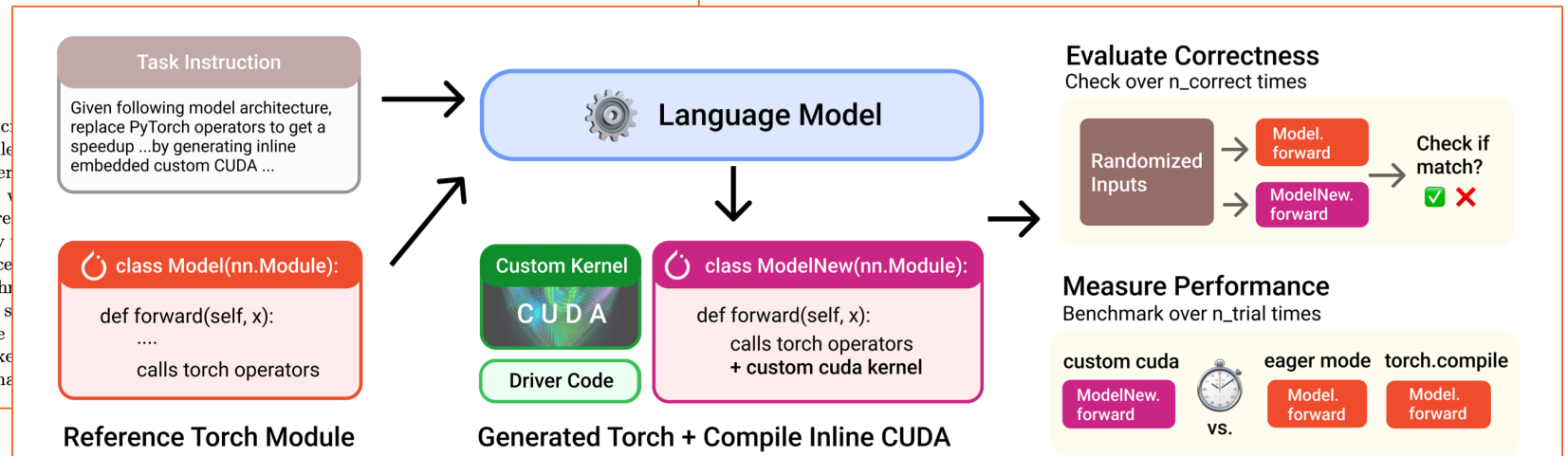
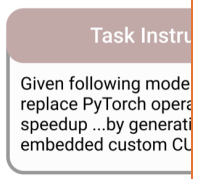
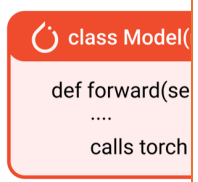


Figure 1: **KernelBench evaluates LMs' ability to generate performant GPU Kernels**. Overview of tasks in KernelBench: KernelBench tasks LMs with generating optimized CUDA kernels for a given target PyTorch model architecture and conducts automated evaluation

KernelBench: Can LLMs Write Efficient GPU Kernels?

Anne Ouyang[1,*], Simon Guo





Reference Tor

Figure 1: **Ker**
tasks in Kerne
PyTorch mode

3.2 Task Selection

The 250 tasks in KernelBench are partitioned into three levels, based on the number of primitive operations, or PyTorch library functions, they contain:

- **Level 1 (100 tasks): Single primitive operation.** This level includes the foundational building blocks of AI (e.g. convolutions, matrix-vector and matrix-matrix multiplications, losses, activations, and layer normalizations).

  Since PyTorch makes calls to several well-optimized and often closed-source kernels under-the-hood, it can be challenging for LMs to outperform the baseline for these primitive operations. However, if an LM succeeds, the open-source kernels could be an impactful alternative to the closed-source (e.g., CuBLAS [27]) kernels.

- **Level 2 (100 tasks): Operator sequences.** This level includes AI workloads containing multiple primitive operations, which can be fused into a single kernel for improved performance (e.g., a combination of a convolution, ReLU, and bias).

  Since compiler-based tools such as the PyTorch compiler are effective at fusion, it can be challenging for LMs to outperform them. However, LMs may propose more complex algorithms compared to compiler rules.

- **Level 3 (50 tasks): Full ML architectures.** This level includes architectures that power popular AI models, such as AlexNet and MiniGPT, collected from popular PyTorch repositories on GitHub.

  Given the scale of modern models, it is critical to use kernels when running training and inference. Unfortunately, it has been difficult for the AI community to generate performant kernels. For instance, it took 5 years from the release of the Transformer architecture [39] to obtain performant kernels [8], let alone today's many new architectures. Peak performance kernels for these architectures require algorithmic modifications that are often beyond the scope of a compiler.

We reiterate that each task contains a meaningful set of AI primitive operations or architectures, such that LM success on the task can directly lead to real world impact.

4.1 One-shot Baseline

We evaluate LMs using a prompt that contains one example of a PyTorch `Model` input and `ModelNew` output, highlighting the task format. The example is simple, containing only an `add` operator (See Appendix C.1). Given this in-context example and the PyTorch task `Model` to optimize, the LM generates `ModelNew` via greedy decoding. We profile the generated code on an NVIDIA L40S GPU, and measure the fast_p metric across all problems. Table 1 shows that the LM-generated kernels achieves a speedup over PyTorch Eager in fewer than 20% of tasks on average.

fast_1 over:	PyTorch Eager			torch.compile		
KernelBench Level	1	2	3	1	2	3
GPT-4o	4%	5%	0%	18%	4%	**4%**
OpenAI o1	10%	24%	**12%**	28%	19%	**4%**
DeepSeek V3	6%	4%	8%	20%	2%	2%
DeepSeek R1	**12%**	**36%**	2%	**38%**	**37%**	2%
Claude 3.5 Sonnet	10%	7%	2%	29%	2%	2%
Llama 3.1-70B Inst.	3%	0%	0%	11%	0%	0%
Llama 3.1-405B Inst.	3%	0%	2%	16%	0%	0%

Table 1: **KernelBench is a challenging benchmark for current LMs**. Here we present fast_1, i.e. the percentage of problems where the model-generated kernel is faster than the PyTorch Eager and `torch.compile` baseline (default configuration) on NVIDIA L40S.

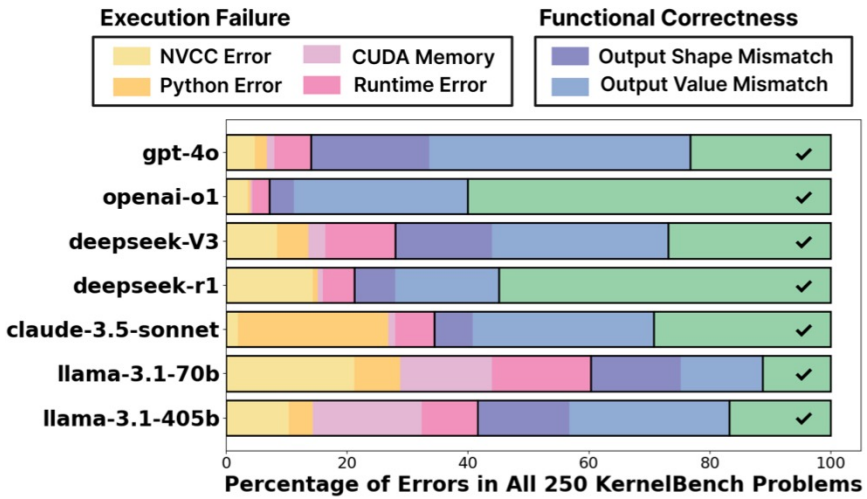


Figure 2: **We categorize failure modes of kernel code into execution failure and functional correctness**. For the one-shot baseline, reasoning models generate fewer kernels with execution failures, but all models struggle similarly with functional correctness.

KernelBench-X: A Comprehensive Benchmark for Evaluating LLM-Generated GPU Kernels

Han Wang* **Jintao Zhang*** **Kai Jiang** **Haoxu Wang** **Jianfei Chen** **Jun Zhu**

https://github.com/BonnieW05/KernelBenchX

KernelBench-X: A Comprehensive Benchmark for Evaluating LLM-Generated GPU Kernels

Han Wang* Jir

2.1 Hardware Efficiency Metrics

Hardware efficiency describes how effectively a kernel uses the available hardware resources. Beyond runtime, it provides a more direct view of whether performance is close to the hardware limit. For a kernel k, let $T(k)$ denote the measured runtime, $B(k)$ the total bytes moved, and $F(k)$ the total floating-point operations. We define the achieved bandwidth and achieved throughput as

$$\mathrm{BW}(k) = \frac{B(k)}{T(k)}, \qquad \mathrm{TP}(k) = \frac{F(k)}{T(k)}, \tag{1}$$

reported in GB/s and TFLOPS respectively. To enable comparison across hardware, we normalize these quantities by their corresponding peak values:

$$\mathrm{IOU}(k) = \frac{\mathrm{BW}(k)}{\mathrm{BW}_{\max}}, \qquad \mathrm{MFU}(k) = \frac{\mathrm{TP}(k)}{\mathrm{TP}_{\max}}. \tag{2}$$

IOU measures memory bandwidth utilization, while MFU measures compute utilization. They are complementary, as some kernels are memory-bound while others are compute-bound.

Wang et al., KernelBench-X: A Comprehensive Benchmark for Evaluating LLM-Generated GPU Kernels, 2026 — github.com/BonnieW05/KernelBenchX

KernelBench-X: A Comprehensive Benchmark for Evaluating LLM

Han

2.1 Hardware Efficiency Metrics

Hardware efficiency describes how effectively a kernel uses the available hardware resources. Beyond runtime, kernel *k* floating-

reported these qu

IOU me complen

Table 1: Overall results on 176 tasks. Speedup and score are averaged over correct kernels only. Correct/Compile highlights how often compiled candidates actually preserve semantics.

Method	Compile (%)	Correct (%)	Correct/Compile (%)	Speedup	Score (%)
AutoTriton	36.4	17.0	46.9	1.35	60.7
GEAK	68.8	30.7	44.6	1.15	50.0
KernelAgent	64.2	10.8	16.8	1.41	68.1
Claude	45.5	22.7	50.0	1.26	54.2
KernelLLM	1.7	0.0	0.0	–	0.0

Note that FLOP- and byte-based quantities are derived from a fixed task-level target model (the intended computation under an idealized implementation), and should be interpreted as normalized efficiency proxies rather than measurements of the actual executed instructions.

Wang et al., KernelBench-X: A Comprehensive Benchmark for Evaluating LLM-Generated GPU Kernels, 2026 — github.com/BonnieW05/KernelBenchX

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao[†], Daniel Y. Fu[†], Stefano Ermon[†], Atri Rudra[‡], and Christopher Ré[†]

[†]Department of Computer Science, Stanford University
[‡]Department of Computer Science and Engineering, University at Buffalo, SUNY

{trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu, chrismre@cs.stanford.edu

June 24, 2022

Dao et al., FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, NeurIPS 2022 — arXiv:2205.14135

FLASHATTENTION: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao[†], Dani

[†]Dep
[‡]Department of C
{trid,danfu}@

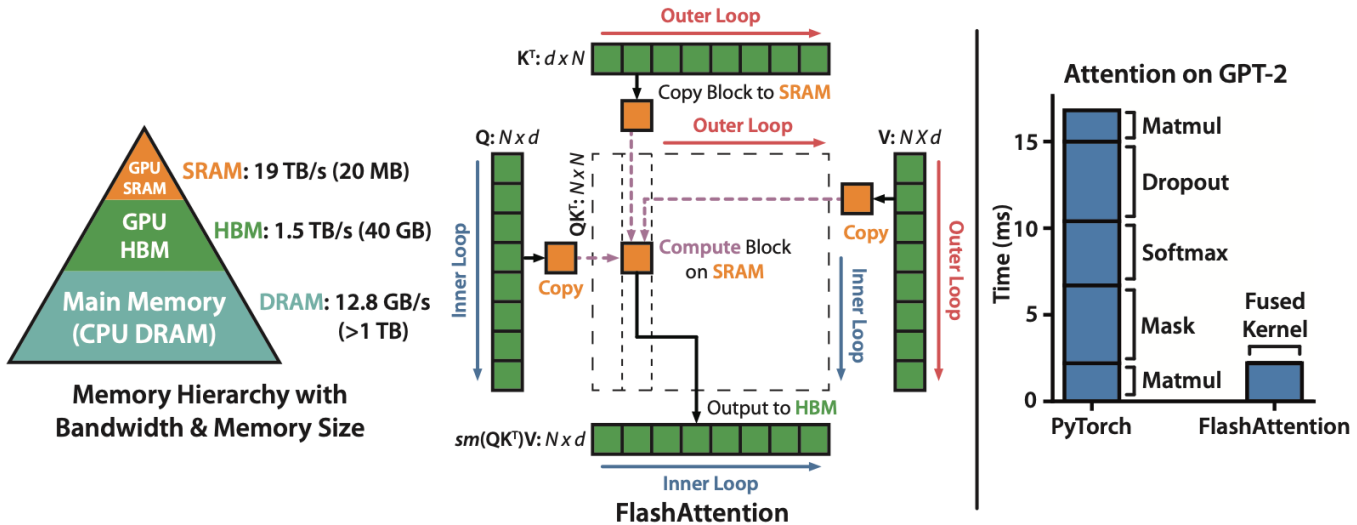
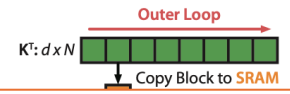


Figure 1: **Left:** FLASHATTENTION uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FLASHATTENTION loops through blocks of the **K** and **V** matrices and loads them to fast on-chip SRAM. In each block, FLASHATTENTION loops over blocks of **Q** matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large $N \times N$ attention matrix to HBM, resulting in an 7.6× speedup on the attention computation.

Dao et al., FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, NeurIPS 2022 — arXiv:2205.14135

FLASHATTENTION: Fast and Memory-Efficient Exact Attention





‡D

Table 2: GPT-2 small and medium using FLASHATTENTION achieve up to 3× speed up compared to Huggingface implementation and up to 1.7× compared to Megatron-LM. Training time reported on 8×A100s GPUs.

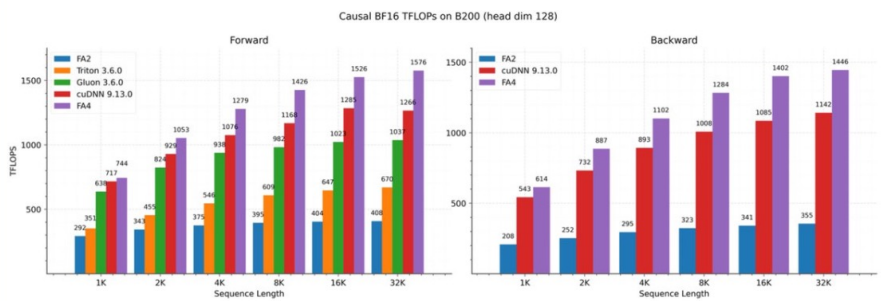
Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FLASHATTENTION	18.2	**2.7 days (3.5×)**
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FLASHATTENTION	14.3	**6.9 days (3.0×)**

Figure 1:
(dotted b
blocks of
loops over blocks of **Q** matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large $N \times N$ attention matrix to HBM, resulting in an 7.6× speedup on the attention computation.

Tri Dao ✓
@tri_dao

The FA4 paper is finally out after a year of work. On Blackwell GPUs, attention now goes about as fast as matmul even though the bottlenecks are so different! Tensor cores are now crazy fast that attn fwd is bottlenecked by exponential, and attn bwd is bottlenecked by shared memory bandwidth.

Some fun stuff in the redesigned algorithm to overcome these bottlenecks: exponential emulation with polynomials, new online softmax to avoid 90% of softmax rescaling, 2CTA MMA instructions that allow two thread blocks to share operands to reduce smem traffic.

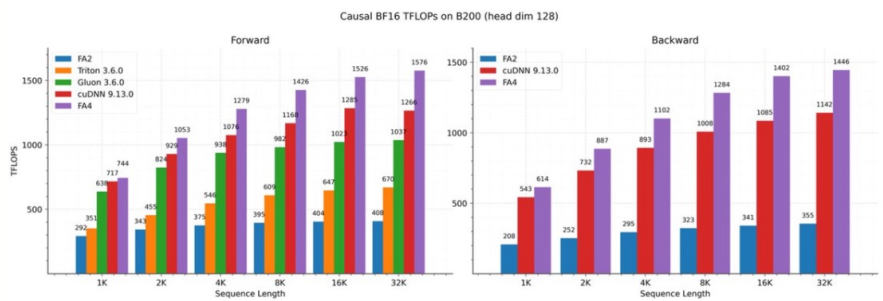
> **Ted Zadouri** ✓ @tedzadouri · Mar 5
>
> Asymmetric hardware scaling is here. Blackwell tensor cores are now so fast, exp2 and shared memory are the wall. FlashAttention-4 changes the algorithm & pipeline so that softmax & SMEM bandwidth no longer dictate speed. Attn reaches ~1600 TFLOPs, pretty much at matmul speed!
>
> FLASHATTENTION-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling
>
> Ted Zadouri[*1,6] Markus Hoehnerbach[*2] Jay Shah[*3]
> Timmy Liu[4] Vijay Thakkar[2,5] Tri Dao[1,6]
> [1]Princeton University [2]Meta [3]Colfax Research [4]NVIDIA [5]Georgia Tech [6]Together AI
>
> 
>

9:49 AM · Mar 5, 2026 · **191.6K** Views

Tri Dao ✓
@tri_dao

The FA4 paper is finally out after a year of work. On Blackwell GPUs, attention now goes about as fast as matmul even though the bottlenecks are so different! Tensor cores are now crazy fast that attn fwd is bottlenecked by exponential, and attn bwd is bottlenecked by shared memory bandwidth.

Some fun stuff in the redesigned algorithm to overcome these bottlenecks: exponential emulation with polynomials, new online softmax to avoid 90% of softmax rescaling, 2CTA MMA instructions that allow two thread blocks to share operands to reduce smem traffic.

> **Ted Zadouri** ✓ @tedzadouri · Mar 5
> Asymmetric hardware scaling is here. Blackwell tensor cores are now so fast, exp2 and shared memory are the wall. FlashAttention-4 changes the algorithm & pipeline so that softmax & SMEM bandwidth no longer dictate speed. Attn reaches ~1600 TFLOPs, pretty much at matmul speed!
>
> FLASHATTENTION-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling
>
> Ted Zadouri[*1,6] Markus Hoehnerbach[*2] Jay Shah[*3]
> Timmy Liu[4] Vijay Thakkar[2,5] Tri Dao[1,6]
> [1]Princeton University [2]Meta [3]Colfax Research [4]NVIDIA [5]Georgia Tech [6]Together AI
>
> 
>

9:49 AM · Mar 5, 2026 · **191.6K** Views

Tri Dao ✓ @tri_dao · Mar 5
I'm unreasonably excited about the fact that we wrote everything in Cute-DSL, embedded in Python. Installing / "compiling" now takes seconds instead of minutes / hours (looking at you, C++ templates). Try pip install fa4!

4 28 426 28K

Tri Dao ✓ @tri_dao · Mar 5
Claude / Codex also have an easier time writing some components of FA4 thanks to the fast compile time. I got Claude to debug a deadlock when we first implemented 2CTA fwd. It ran autonomously overnight for 6 hours, figured out part of the fix, but then went down a rabbit hole convincing itself that the compiler is broken (so very human 😂). After 6 hours, from Claude's partial fix, I was able to fix the hang in 10 mins. More details here: github.com/Dao-AILab/flas...
I'm hoping FA5 will be written completely by AI

> **Dao-AILab/flash-attention**
> Fast and memory-efficient exact attention
> 238 Contributors · 1k Issues · 25 Discussions · 25k Stars · 3k Forks
> flash-attention/AI/DEBUG_2CTA.md at main · Dao-AILab/flash-attention

From github.com

2 26 339 30K

Zadouri, Hoehnerbach, Shah, Thakkar, Dao — FlashAttention-4: Algorithm and Kernel Pipelining Co-Design, MLSys 2026

Google DeepMind

AlphaEvolve: A coding agent for scientific and algorithmic discovery

Alexander Novikov*, Ngân Vũ*, Marvin Eisenberger*, Emilien Dupont*, Po-Sen Huang*, Adam Zsolt Wagner*, Sergey Shirobokov*, Borislav Kozlovskii*, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli and Matej Balog*
Google DeepMind[1]

In this white paper, we present *AlphaEvolve*, an evolutionary coding agent that substantially enhances capabilities of state-of-the-art LLMs on highly challenging tasks such as tackling open scientific problems or optimizing critical pieces of computational infrastructure. *AlphaEvolve* orchestrates an autonomous pipeline of LLMs, whose task is to improve an algorithm by making direct changes to the code. Using an evolutionary approach, continuously receiving feedback from one or more evaluators, *AlphaEvolve* iteratively improves the algorithm, potentially leading to new scientific and practical discoveries. We demonstrate the broad applicability of this approach by applying it to a number of important computational problems. When applied to optimizing critical components of large-scale computational stacks at Google, *AlphaEvolve* developed a more efficient scheduling algorithm for data centers, found a functionally equivalent simplification in the circuit design of hardware accelerators, and accelerated the training of the LLM underpinning *AlphaEvolve* itself. Furthermore, *AlphaEvolve* discovered novel, provably correct algorithms that surpass state-of-the-art solutions on a spectrum of problems in mathematics and computer science, significantly expanding the scope of prior automated discovery methods (Romera-Paredes et al., 2023). Notably, *AlphaEvolve* developed a search algorithm that found a procedure to multiply two 4×4 complex-valued matrices using 48 scalar multiplications; offering the first improvement, after 56 years, over Strassen's algorithm in this setting. We believe *AlphaEvolve* and coding agents like it can have a significant impact in improving solutions of problems across many areas of science and computation.

Novikov et al., AlphaEvolve: A coding agent for scientific and algorithmic discovery, Google DeepMind 2025 — arXiv:2506.13131

Google DeepMind

AlphaEv
algorith

Alexander Novikov
Sergey Shiroboko
See, Swarat Chau
Google DeepMind[1]

In this white pa
capabilities of st
or optimizing cr
pipeline of LLM
an evolutionary
iteratively impr
demonstrate th
putational prob
stacks at Google
a functionally e
ated the trainin
novel, provably
in mathematics
methods (Romer
procedure to mu
first improveme
coding agents lil
of science and c

We challenged *AlphaEvolve* to directly optimize the XLA-generated IRs encapsulating the FlashAttention kernel along with pre- and postprocessing code. We optimized a configuration corresponding to a highly impactful transformer model used for inference at scale on GPUs, with the goal of minimizing the module's overall execution time. This was a particularly challenging task, because (1) the IR is designed for debugging purposes rather than for direct editing by developers, and (2) it is compiler-generated and already highly optimized. Each modification proposed by *AlphaEvolve* was checked against the reference (unmodified) code on randomized inputs in order to ensure numerical correctness throughout optimization. The final version of the code was rigorously confirmed by human experts to be correct for all possible inputs.

AlphaEvolve was able to provide meaningful optimizations for both levels of abstraction exposed by the IR. Firstly, the FlashAttention kernel for the configuration of interest was sped up by 32%. Secondly, *AlphaEvolve* found improvements in pre- and postprocessing of kernel inputs and outputs, resulting in a 15% speed up in this part. These results demonstrate the ability of *AlphaEvolve* to optimize compiler-generated code, offering the potential of incorporating discovered optimizations into existing compilers for specific use cases, or, in the longer term, incorporating *AlphaEvolve* into the compiler workflow itself.

Novikov et al., AlphaEvolve: A coding agent for scientific and algorithmic discovery, Google DeepMind 2025 — arXiv:2506.13131

Google DeepMind

AlphaEv
algorith

Alexander Novikov
Sergey Shiroboko
See, Swarat Chau
Google DeepMind[1]

In this white pa
capabilities of st
or optimizing cr
pipeline of LLM
an evolutionary
iteratively impr
demonstrate th
putational prob
stacks at Google
a functionally e
ated the trainin
novel, provably
in mathematics
methods (Romer
procedure to mu
first improveme
coding agents lik
of science and c

We challenged *AlphaEvolve* to directly optimize the XLA-generated IRs encapsulating the FlashAttention kernel along with pre- and postprocessing code. We optimized a configuration corresp
with th
challeng
editing
modific
on rand
The fina
possible

Alph
exposed
up by 3
inputs

the ability of *AlphaEvolve* to optimize compiler-generated code, offering the potential of incorporating discovered optimizations into existing compilers for specific use cases, or, in the longer term, incorporating *AlphaEvolve* into the compiler workflow itself.

Hacker News new | past | comments | ask | show | jobs | submit

▲ jasonjmcghee on May 14, 2025 | parent | context | favorite | on: AlphaEvolve: A Gemini-powered coding agent for des...

> AlphaEvolve achieved up to a 32.5% speedup for the FlashAttention kernel implementation in Transformer-based AI models

> In roughly 75% of cases, it rediscovered state-of-the-art solutions, to the best of our knowledge.

> And in 20% of cases, AlphaEvolve improved the previously best known solutions

These sound like incredible results. I'd be curious what kind of improvements were made / what the improvements were.

Like, was that "up to a 32.5% speedup" on some weird edge case and it was negligible speed up otherwise? Would love to see the benchmarks.

▲ schmidtleonard on May 14, 2025 | next [–]

Remember that GPUs have cache hierarchies and matching block sizes to optimally hit those caches is a big win that you often don't get by default, just because the number of important kernels times important GPUs times effort to properly tune one is greater than what people are willing to do for others for free in open source. Not to mention kernel fusion and API boundaries that socially force suboptimal choices for the sake of clarity and simplicity.

It's a very impressive result, but not magic, but also not cheating!

▲ hiddencost on May 14, 2025 | parent | next [–]

100%. LLMs are extremely useful for doing obvious but repetitive optimizations that a human might miss.

▲ jerjerjer on May 14, 2025 | root | parent | next [–]

What it essentially does is a debugging/optimization loop where you change one thing, eval, repeat it again and compare results.

Previously we needed to have a human in the loop to do the change. Of course we have automated hyperparameter tuning (and similar things), but that only works only in a rigidly defined search space.

Will we see LLMs generating new improved LLM architectures, now fully incomprehensible to humans?

Novikov et al., AlphaEvolve: A coding agent for scientific and algorithmic discovery, Google DeepMind 2025 — arXiv:2506.13131

Validation-Centric AI-Assisted GPU Porting of a 250,000+ Line Legacy Weather Simulation Code

Tetsuya Hoshino
Information Technology Center
Nagoya University
Aichi, Japan
hoshino@cc.nagoya-u.ac.jp

Masaya Kato
Institute for Space-Earth Environmental Research
Nagoya University
Aichi, Japan

Kazuhisa Tsuboki
Institute for Space-Earth Environmental Research
Nagoya University
Aichi, Japan

Daichi Mukunoki
Information Technology Center
Nagoya University
Aichi, Japan

Takahiro Katagiri
Information Technology Center
Nagoya University
Aichi, Japan

Toshihiro Hanawa
Information Technology Center
The University of Tokyo
Chiba, Japan

Hoshino et al., Validation-Centric AI-Assisted GPU Porting of a 250,000+ Line Legacy Weather Simulation Code, 2026

Validation-Centric AI-Assisted GPU Porting of a 250,000+ Line Legacy Weather Simulation Code

Tetsuya Hoshino
Information Technology Center
Nagoya University
Aichi, Japan
hoshino@cc.nagoya-u.ac.jp

Masaya Kato
Institute for Space-Earth Environmental Research
Nagoya University
Aichi, Japan

Kazuhisa Tsuboki
Institute for Space-Earth Environmental Research
Nagoya University
Aichi, Japan

Daichi Mukunoki
Information Technology Center
Nagoya University
Aichi, Japan

Takahiro Katagiri
Information Technology Center
Nagoya University
Aichi, Japan

Toshihiro Hanawa
Information Technology Center
The University of Tokyo
Chiba, Japan

An Ocean Model Ported by a Large Language Model: Experience and Lessons from FESOM2 (Fortran to C to C++/Kokkos)

N. Koldunov[1], S. K. Cheedela[1], S. Danilov[1], D. Sidorenko[1], S. Beyer[1], and T. Jung[1,2]

[1]Alfred Wegener Institute, Helmholtz Centre for Polar and Marine Research, Bremerhaven, Germany
[2]Department of Physics and Electrical Engineering, University of Bremen, Bremen, Germany

June 11, 2026

Koldunov et al., An Ocean Model Ported by a Large Language Model: Experience and Lessons from FESOM2 (Fortran → C → C++/Kokkos), 2026

Validation-Centric AI-Assisted GPU Porting of a 250,000+ Line Legacy Weather Simulation Code

Tetsuya Hoshino
Information Technology Center
Nagoya University
Aichi, Japan
hoshino@cc.nagoya-u.ac.jp

Masaya Kato
Institute for Space-Earth Environmental Research
Nagoya University
Aichi, Japan

Kazuhisa Tsuboki
Institute for Space-Earth Environmental Research
Nagoya University
Aichi, Japan

Daichi Mukunoki
Information Technology Center
Nagoya University
Aichi, Japan

Takahiro Katagiri
Information Technology Center
Nagoya University
Aichi, Japan

From Legacy Fortran to Portable Kokkos: An Autonomous Agentic AI Workflow

Sparsh Gupta, Kamalavasan Kamalakkannan, Maxim Moraru, Galen Shipman, and Patrick Diehl

Experience
/Kokkos)

N. Koldunov[1], S. K. Cheedela[1], S. Danilov[1], D. Sidorenko[1], S. Beyer[1], and T. Jung[1,2]

[1]Alfred Wegener Institute, Helmholtz Centre for Polar and Marine Research, Bremerhaven, Germany
[2]Department of Physics and Electrical Engineering, University of Bremen, Bremen, Germany

June 11, 2026

Gupta et al., From Legacy Fortran to Portable Kokkos: An Autonomous Agentic AI Workflow, 2026

From Legacy Fortran to Portable Kokkos: An A

Sparsh Gupta, Ka

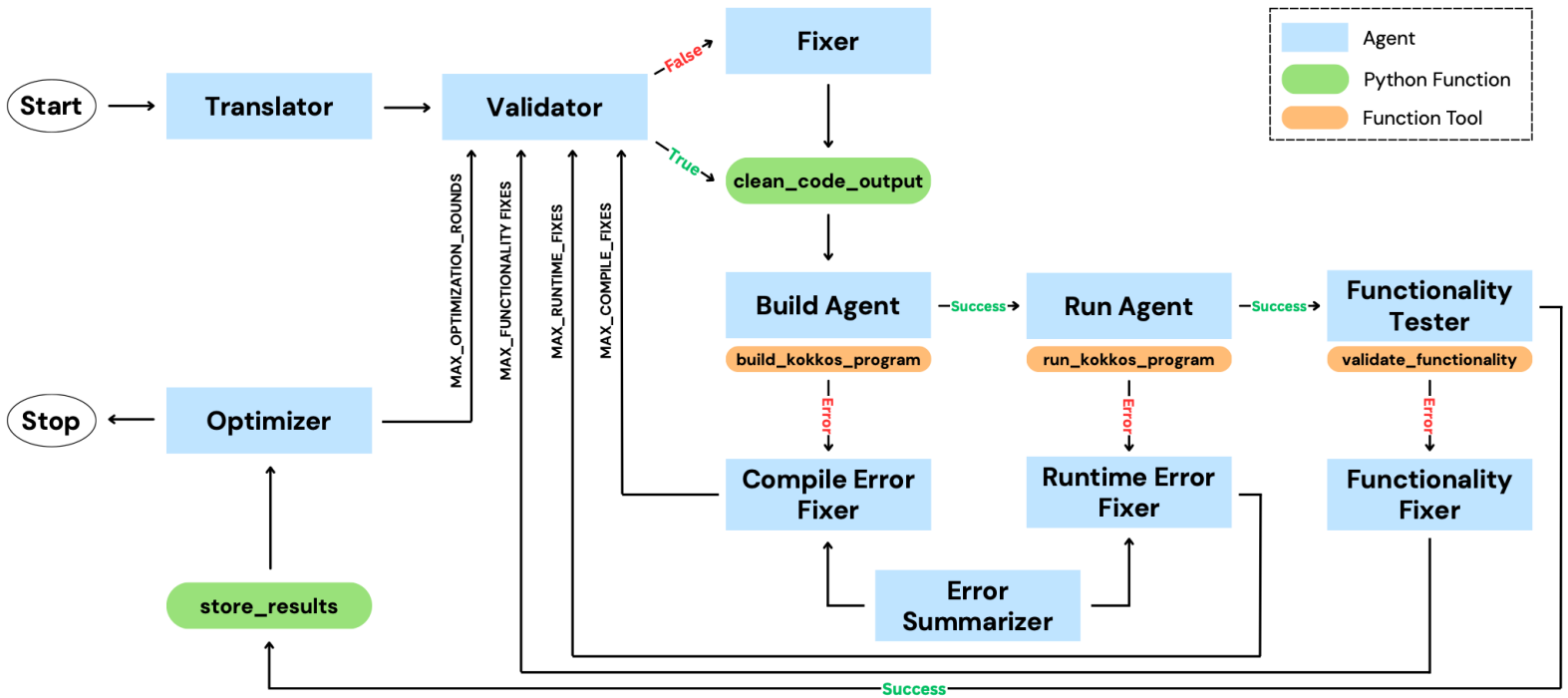


Figure 1. Agentic AI workflow for autonomous Fortran-to-Kokkos translation, validation, compilation, runtime execution, functionality testing, and performance optimization. Fixer Agents are triggered on error events (*e.g.*, failed compilation, runtime fault, or incorrect functionality testing output). Agent invocation limits at each stage are enforced via configurable thresholds (*e.g.*, `MAX_COMPILE_FIXES`). Function tools invoked by the Build and Run agents (`build_kokkos_program`, `run_kokkos_program`) utilize SLURM to schedule and monitor jobs and Spack to load the correct Kokkos environment on the hardware partitions. All artifacts and metrics are versioned and stored per version run using the `store_results` function.

Gupta et al., From Legacy Fortran to Portable Kokkos: An Autonomous Agentic AI Workflow, 2026

From Lega

Spars

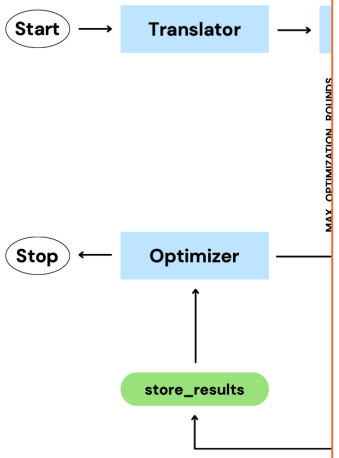


Figure 1. Agentic AI workflow for autonomou
optimization. Fixer Agents are triggered on erro
each stage are enforced via configurable thresh
`run_kokkos_program`) utilize SLURM to s
artifacts and metrics are versioned and stored

D. Optimization Evaluation

To compare optimization progress across versions, we report GFLOPS at the maximum input size for the five kernels with closed-form FLOP models, and wall-clock speedup for biharmonic (explained below). GFLOPS is computed as:

$$GFLOPS = \frac{FLOPS(n, \hat{i}, r)}{t_{kernel} \times 10^9}, \tag{3}$$

where n is the input size, $\hat{i}$ is the effective per-size iteration count after scaling, r is the kernel repetitions per run, and t_{kernel} is the wall-clock runtime over the full loop. FLOPS use closed-form models calibrated to NPB-3 class-A conventions where applicable:

- CG: sparse SpMV on the `makea()` matrix ($nnz = 43n$, inner-CG cap $c_{max} = 25$); FLOPS $= r(2\,nnz + 3n + c_{max}(2\,nnz + 10n)) + (2\,nnz + 3n)$.
- EP: polar method (RNG plus $\pi/4$ accept rate); FLOPS $= r(50 \cdot 2^{n+1} + 8(\frac{\pi}{4})2^n) \approx 106\,r \cdot 2^n$, excluding transcendentals.
- FT: FLOPS $= r(5n^3 \log_2 n^3)$.
- MG: V-cycle stencil work; FLOPS $= r(576\,n^3)$.

Gupta et al., From Legacy Fortran to Portable Kokkos: An Autonomous Agentic AI Workflow, 2026